



OSTİM TECHNICAL UNIVERSITY FACULTY OF ENGINEERING

ANTI-MONEY LAUNDERING DETECTION SYSTEM

**Moavia Mahmood
210208693**

**Project Advisor:
Assist. Prof. Dr. Yücel Tekin**

**Undergraduate Project
Software Engineering
MFBP 402 (1) [360637] Graduation Project II**

May 2026

APPROVAL

This undergraduate project, entitled “Anti-Money Laundering Detection System” prepared by Moavia Mahmood under the supervision of Asst. Prof. Dr. Yücel Tekin, has been examined by the committee and accepted as an undergraduate project in terms of scope and quality.

Asst. Prof. Dr. Yücel Tekin
Supervisor

.....

Prof. Dr. Jury Member Name Surname
Member

.....

Assoc. Prof. Dr. Jury Member Name Surname
Member

.....

Asst. Prof. Dr. Yücel Tekin
Head of Department

.....

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor for their continuous guidance, support, and valuable feedback throughout the development of this project. Their expertise and encouragement played a crucial role in shaping the direction and quality of this work.

I am also thankful to the faculty members of the Software Engineering department for providing the knowledge and academic foundation necessary to carry out this research. Their courses and insights significantly contributed to my understanding of data engineering and real-time systems.

Furthermore, I would like to acknowledge my friends for their support, discussions, and motivation during the course of this project. Their encouragement helped me overcome challenges and stay focused.

Finally, I am deeply grateful to my family for their unwavering support, patience, and belief in me throughout my academic journey. Without their encouragement, this work would not have been possible.

ABSTRACT

Money laundering remains one of the most critical threats to global financial integrity, with the United Nations Office on Drugs and Crime estimating annual illicit flows equivalent to 2–5% of global GDP. Traditional Anti-Money Laundering (AML) systems rely on overnight batch processing and static rule-based engines, imposing detection latency of eight to twenty-four hours and generating false-positive rates that consistently exceed ninety percent. This project presents the design, implementation, and empirical evaluation of a scalable, serverless, event-driven AML data engineering pipeline deployed entirely on Amazon Web Services (AWS).

The system employs Amazon Kinesis Data Streams for real-time transaction ingestion, AWS Lambda for stateless and stateful stream processing, and Amazon DynamoDB for per-customer rolling windows that support temporal detection rules. All raw and processed data are persisted in a Hive-style date-partitioned Amazon S3 data lake, catalogued through AWS Glue, and exposed via a set of Amazon Athena cleaned views for consistent SQL-based analytical querying. A FastAPI REST backend and a React/TypeScript operational dashboard SENTINEL complete the end-to-end pipeline, providing compliance analysts with real-time KPIs, alert breakdowns, and entity risk profiles.

Fourteen rule-based AML detection patterns are implemented across the canonical placement, layering, and integration phases of money laundering, covering structuring, cross-border high-value transfers, rapid fund movement, shell-company routing, round-trip transactions, and impossible-travel anomalies. Empirical evaluation demonstrates a median end-to-end latency of 8–11 seconds from transaction generation to queryable alert, a reduction of more than three orders of magnitude compared to legacy batch systems. The project additionally documents six data quality patterns that arise when streaming transactional data into a partitioned data lake alongside slowly-changing customer and account dimensions, and demonstrates their resolution at the query layer using Athena window functions and partition projection rather than at ingestion time.

The findings confirm that a fully serverless, managed-service architecture on AWS can sustain sub-minute AML detection latency without the operational overhead of self-managed streaming clusters, while remaining accessible for academic prototyping and small-scale institutional deployment.

Keywords: Anti-Money Laundering (AML), Real-Time Stream Processing, Serverless Architecture, Amazon Kinesis, AWS Lambda, Amazon DynamoDB, Data Lake, Amazon S3, Amazon Athena, Financial Crime Detection, Slowly-Changing Dimensions, Event-Driven Architecture, Data Engineering, Rule-Based Detection, Cloud-Native Systems

TABLE OF CONTENT

APPROVAL.....	2
ACKNOWLEDGEMENTS.....	3
ABSTRACT.....	4
TABLE OF CONTENT.....	5
LIST OF TABLES.....	7
LIST OF FIGURES.....	8
LIST OF SYMBOLS AND ABBREVIATIONS.....	9
1. Introduction.....	10
1.1 Background and Motivation.....	10
1.2 Problem Statement.....	10
1.3 Research Questions.....	11
1.4 Aim and Objectives.....	11
1.5 Scope and Limitations.....	11
1.6 Contribution.....	12
1.7 Report Structure.....	12
2. General Concepts.....	13
2.1. Domain Definition.....	13
2.2. Technologies Used.....	13
2.3. Fundamental Concepts.....	14
3. Related Work.....	17
3.1. Existing Systems.....	17
3.2. Academic Studies.....	17
3.3. Comparative Analysis.....	19
4. Software Process Model.....	21
4.1. Selected Software Development Model.....	21
4.2. Definition of Scrum Roles.....	21
4.3. Scrum Process.....	22
4.4. Scrum Implementation in the Project.....	23
5. Requirements Analysis.....	26
5.1. Stakeholders.....	26
5.2. User Requirements.....	26
5.2.1. Academic Stakeholders (Directly Involved):.....	26
5.2.2. Theoretical Stakeholders (System End-Users & Beneficiaries):.....	26
5.2.3. Functional Requirements.....	27
5.2.4. Non-Functional Requirements.....	28
5.3. System Specifications.....	30
5.3.1. Local Development Environment Specifications.....	30
5.3.2. Cloud Infrastructure Specifications (AWS).....	30
5.3.3. Network and Security Specifications.....	32

6. System Modeling.....	33
6.1. Context Diagram.....	33
6.2. Use Case Diagram.....	34
6.3. Use Case Descriptions.....	36
6.4. Sequence Diagrams.....	36
7. System Architecture and Design.....	39
7.1. System Architecture.....	39
7.2. Structural Models.....	41
7.2.1 Data Model (Entity-Relationship Diagram).....	41
7.2.2. Inheritance (Generalization).....	41
7.2.3. Aggregation / Composition.....	42
7.3. Behavioral Models.....	42
7.3.1. Event-Driven Model.....	42
7.3.2. State Diagram.....	43
8. System Implementation.....	44
8.1. Development Environment.....	44
8.2. System Components.....	44
8.3. Code Structure.....	45
9. MVP Development.....	49
9.1. MVP Definition.....	49
9.2. Implemented Features.....	49
9.3. System Interfaces.....	50
9.4. System Demonstration.....	53
10. Testing and Evaluation.....	55
10.1. Testing Strategy.....	55
10.2. Test Scenarios.....	55
10.3. Test Results.....	57
10.4. Detection Performance Evaluation.....	59
10.4.1. Evaluation Methodology.....	59
10.4.2. Aggregate Classification Performance.....	60
10.4.3. Per-Pattern Recall.....	61
10.4.4. Per-Rule Precision.....	63
11. Results and Discussion.....	65
12. Conclusion and Future Work.....	67
12.1. Conclusion.....	67
12.2. Future Work.....	67
REFERENCES.....	69
CURRICULUM VITAE.....	70
13.1. Appendix A: Source Code.....	71
13.2. Appendix B: Sprint Backlog.....	71
13.3. Appendix C: User Guide.....	74

LIST OF TABLES

Table 1 : Technologies Used

Table 2 : Comparative Analysis

Table 3 : Use Case Descriptions

Table 4: AML Pattern

Table 5: Data Quality Issues Identified and Resolved

Table 6: End-to-End Latency Measurements

Table 7: Aggregate classification metrics

Table 8: Per-pattern recall

Table 9: Per-rule precision

Table 10: Threshold sensitivity sweep

Table 11: Sprint 1

Table 12: Sprint 2

Table 13: Sprint 3

Table 14: Sprint 4

Table 15: Sprint 5

Table 16: Sprint 6

LIST OF FIGURES

Figure 1: Context Diagram

Figure 2: Use Case Diagram

Figure 3: Sequence Diagram

Figure 4: System Architecture

Figure 5: Class Diagram

Figure 6: Event-Driven Model

Figure 7: State Diagram

Figure 8: Overview

Figure 9: Alerts

Figure 10: Transactions

Figure 11: Entities

Figure 12: FastAPI REST API

Figure 13: Athena Query

Figure 14: Confusion matrix

Figure 15: Recall per AML Pattern

Figure 16: Precision per Detection Rule

LIST OF SYMBOLS AND ABBREVIATIONS

AML	: Anti-Money Laundering
API	: Application Programming Interface
AWS	: Amazon Web Services
BDDK	: Banking Regulation and Supervision Agency
CSV	: Comma-Separated Values
CSDT	: Cost-Sensitive Decision Trees
CSLR	: Cost-Sensitive Logistic Regression
DDB	: DynamoDB
DXA	: Document Extension Attribute (page measurement)
EB	: EventBridge
ETL	: Extract, Transform, Load
FATF	: Financial Action Task Force
FinTech	: Financial Technology
FL	: Federated Learning
FPR	: False Positive Rate
FR	: Functional Requirement
IAM	: Identity and Access Management
JSONL	: JSON Lines (newline-delimited JSON)
LSTM	: Long Short-Term Memory
MVP	: Minimum Viable Product
NFR	: Non-Functional Requirement
PEP	: Politically Exposed Person
QLSTM	: Quantum Long Short-Term Memory
ROC	: Receiver Operating Characteristic
SAR	: Suspicious Activity Report
SCD	: Slowly Changing Dimension
S3	: Simple Storage Service
SDK	: Software Development Kit
SPA	: Single Page Application
SQL	: Structured Query Language
TLS	: Transport Layer Security
TTL	: Time-To-Live
UML	: Unified Modeling Language
UNODC	: United Nations Office on Drugs and Crime
VPC	: Virtual Private Cloud
VAE	: Variational Autoencoders

1. Introduction

1.1 Background and Motivation

Money laundering is the process by which proceeds of crime are integrated into the legitimate financial system. It is estimated by the United Nations Office on Drugs and Crime (UNODC) to involve flows equivalent to 2–5% of global GDP annually. Financial institutions are obliged under recommendations issued by the Financial Action Task Force (FATF), and in Türkiye under regulations enforced by MASAK and the Banking Regulation and Supervision Agency (BDDK), to monitor customer transactions for suspicious activity and to file Suspicious Activity Reports (SARs) within statutory deadlines. Failure to detect laundering activity within the regulatory window exposes institutions to substantial fines, reputational damage, and in severe cases the revocation of operating licences.

Despite decades of investment, transaction-monitoring systems deployed in production banking environments suffer from two well-documented problems. First, detection latency is dominated by overnight batch processing, meaning that a transaction executed in the morning may not be evaluated against detection rules until the following business day. Second, alert quality is poor: industry estimates consistently place the false-positive rate (FPR) of rule-based AML systems above 90%, generating analyst workloads that are operationally unsustainable and economically wasteful. The combination of high latency and high FPR represents the central engineering challenge that this project addresses.

1.2 Problem Statement

Existing AML transaction-monitoring architectures were designed for an era of end-of-day reconciliation and centralised data warehouses. Modern payment rails are instant transfers, mobile wallets, and cross-border crypto on-ramps, producing transaction velocities and volumes that batch architectures cannot evaluate within regulatory time windows. Re-engineering these pipelines using stream-processing frameworks such as Apache Spark Structured Streaming or Apache Flink is technically feasible but introduces substantial operational overhead: cluster provisioning, checkpoint management, version compatibility, and dedicated DevOps capacity. For mid-tier institutions and academic prototypes, this overhead is prohibitive.

A serverless, event-driven alternative that is built on managed AWS components which promises to deliver near real-time detection without the operational burden of cluster management. However, serverless streaming introduces its own engineering challenges, including stateful processing across stateless function invocations, slowly-changing dimension (SCD) inconsistencies between snapshots written to a data lake, and the absence of native windowing primitives that frameworks such as Flink provide out of the box. Whether these challenges can

be resolved without compromising the latency benefits of the serverless approach is an open architectural question that this project investigates.

1.3 Research Questions

This project is guided by two research questions:

RQ1. Can a fully serverless, event-driven architecture on AWS using Amazon Kinesis Data Streams, AWS Lambda, and Amazon DynamoDB sustain sub-minute end-to-end AML detection latency, where end-to-end latency is defined as the interval between transaction emission and alert availability in the analytics layer?

RQ2. What data-quality patterns arise when streaming transactional data into a partitioned data lake alongside slowly-changing customer and account dimensions, and to what extent can these be mitigated at the query layer using Amazon Athena views rather than at the ingestion layer?

1.4 Aim and Objectives

The aim of this project is to design, implement, and empirically evaluate a serverless data engineering pipeline that ingests synthetic banking data, applies rule-based AML detection in near real time, and exposes alerts for analyst review through a query-layer interface. The specific objectives are:

1. To simulate realistic banking activity: customers, accounts, and transactions on a scheduled cadence, with embedded ground-truth labels for functional validation.
2. To stream transactions through Amazon Kinesis Data Streams and process them in a stateful Lambda consumer that maintains per-customer rolling windows in DynamoDB.
3. To implement fourteen detection rules covering structuring, layering, rapid movement, high-value transfers, and risky-account patterns, with severity scoring on a 0–100 scale.
4. To resolve slowly-changing dimension inconsistencies in the analytics layer using Athena views built on `ROW_NUMBER()` window functions and partition projection.
5. To measure end-to-end latency under controlled conditions and document the data-quality patterns encountered during implementation.

1.5 Scope and Limitations

The project explicitly **includes**: data generation, raw and processed S3 storage, Kinesis-based streaming ingestion, stateful rule-based detection, DynamoDB state management with TTL expiry, Athena views for analytics, and a FastAPI-based query interface for alert review.

The project explicitly **excludes**, and identifies as future work: machine learning models for anomaly detection (e.g., Isolation Forest, autoencoders); independently labelled benchmark

datasets for accuracy evaluation; full deployment of the FastAPI backend to AWS Lambda with API Gateway; graph-analytic detection of cross-customer layering using Amazon Neptune; and integration with real-world payment networks.

The principal limitations of the work are:

1. Detection accuracy is validated against synthetic ground-truth labels generated by the same data generator, which constitutes circular validation rather than independent benchmarking.
2. Latency measurements are conducted under controlled load and do not characterise behaviour at production-scale burst rates.
3. The system has not undergone load, security, or penetration testing.
4. The Kinesis consumer does not currently route failed records to a dead-letter queue; failed batches are retried by Lambda's default behaviour and discarded after the retry budget is exhausted.

1.6 Contribution

The contribution of this project is **architectural rather than algorithmic**. Specifically, the work documents:

1. A stateful serverless streaming pattern (Kinesis $\rightarrow$ Lambda $\rightarrow$ DynamoDB with item-level TTL) suitable for AML use cases without managed stream-processing clusters.
2. A query-layer resolution of slowly-changing dimension inconsistencies in streaming data lakes, using Athena partition projection and window-function views rather than ingestion-time deduplication.
3. An empirical latency profile ($N = 30$ trials) for the proposed architecture, with operational definitions sufficient for replication.

No novel detection algorithm is claimed; the fourteen rules implemented are drawn from documented AML typologies and adapted to the streaming context.

1.7 Report Structure

The remainder of this report is organised as follows. Chapter 2 reviews relevant literature on AML detection systems, serverless data engineering, and stream-processing patterns. Chapter 3 details the methodology, including the development process and requirements engineering. Chapter 4 presents the system architecture and component design. Chapter 5 describes the implementation, including data generation, stream processing, and the analytics layer. Chapter 6 reports the empirical evaluation, including latency measurements and data-quality findings. Chapter 7 discusses the results in the context of the research questions, identifies limitations, and outlines future work. Chapter 8 concludes.

2. General Concepts

2.1. Domain Definition

This project falls within the domains of financial technology (FinTech), data engineering, and cybersecurity, with a specific focus on AML systems. AML refers to the processes, regulations, and technologies used by financial institutions to detect and prevent illegal activities such as money laundering, terrorist financing, and financial fraud. With the rapid growth of digital banking and online payment platforms, the AML domain has become increasingly reliant on advanced data processing and analytical techniques.

The project specifically operates at the intersection of real-time data engineering, serverless cloud architecture, and rule-based fraud detection. It involves processing high-velocity financial transaction streams and applying classification logic across fourteen distinct AML patterns. This requires knowledge of distributed systems, event-driven architectures, partitioned data lakes, query-layer data quality, and modern API/UI design.

The domain includes practical challenges such as data scalability, low-latency processing, slowly-changing dimensions, false-positive management, and operational cost. The project addresses these by integrating modern AWS managed services with a clean separation between raw and curated data layers, demonstrating a production-style approach to AML data engineering.

2.2. Technologies Used

The following technology stack was selected to deliver a fully serverless, low-cost, and operationally simple system. AWS managed services were chosen over self-managed alternatives (such as Apache Kafka or Apache Spark on EC2) to eliminate cluster management, reduce idle-cost, and benefit from native integration between services.

Technology / Tool	Purpose / Role in Project	AWS Service / Alternative
Real-Time Streaming	Ingest high-volume transaction events with ordering guarantees per customer	Amazon Kinesis Data Streams
Event Scheduling	Trigger the data-generation workflow at a fixed cadence	Amazon EventBridge
Serverless Compute	Generate, ingest, and process data without provisioned servers	AWS Lambda (Python 3.11)

Workflow Orchestration	Sequence the three data-generation Lambdas (entity → transaction → batch alert) deterministically per cycle	AWS Step Functions
Data Lake Storage	Store raw and cleaned data with date partitioning	Amazon S3
Stateful Storage for Streaming	Maintain rolling per-customer transaction windows for time-based detection rules	Amazon DynamoDB
Data Catalog	Auto-discover schemas and partitions	AWS Glue Data Catalog + Crawlers
Analytical SQL Layer	Query the data lake with cleaned, typed views	Amazon Athena
Backend API	Expose Athena-backed REST endpoints to the dashboard	FastAPI (Python) + Uvicorn
Frontend	Operational dashboard for compliance analysts	React 19 + TypeScript + Vite
Programming Languages	Implement detection logic, generators, API, UI	Python 3.11 (backend), TypeScript (frontend), SQL (Athena)
Monitoring & Logging	Track Lambda invocations, errors, and Kinesis consumer lag	AWS CloudWatch
Access Management	Enforce least-privilege between services	AWS IAM
Version Control & CI	Source code management and history	Git + GitHub

Table 1: Technologies Used

2.3. Fundamental Concepts

The system relies on several fundamental concepts spanning AML domain knowledge, cloud engineering, and modern data architecture.

Anti-Money Laundering (AML) refers to the set of policies, regulations, and technical procedures used by financial institutions to prevent, detect, and report suspicious financial activities that could be related to money laundering, terrorist financing, or other financial crimes.

AML systems aim to identify unusual transaction patterns, flag suspicious accounts, and support regulatory compliance reporting. This project implements fourteen distinct AML patterns including the three canonical phases of money laundering: placement (cash deposits below reporting thresholds), layering (rapid movement across multiple accounts), and integration (re-introducing funds through trade-based or shell-company channels).

Real-Time Data Processing involves analysing data as it is generated rather than in scheduled batches. In this project, transactions move from generation to alert classification within seconds via Amazon Kinesis Data Streams. Compared to overnight batch processing common in legacy AML systems, this approach eliminates the regulatory exposure window between when a suspicious transaction occurs and when it is flagged.

Serverless Architecture is the design pattern in which compute resources are provisioned, scaled, and billed on a per-invocation basis with no idle capacity. Every compute component in this system (data generators, transaction processor, AML consumer, API backend during demos) follows this model. The architectural benefit is operational simplicity and cost; the system idles at near-zero cost when no data is flowing.

Hive-Style Partitioning is the practice of structuring S3 keys as `prefix/partition_key=value/file.ext` so that downstream query engines (Glue, Athena) automatically interpret the partition column. The project partitions every dataset by date (`dt=YYYY-MM-DD`), which reduces Athena scan size and cost, since queries with `WHERE dt = '...'` only read the relevant day's data.

Stateful Stream Processing. Several AML detection rules require temporal memory: `STRUCTURING_WINDOW` (three or more deposits in the 8–10K band within 30 minutes), `VELOCITY_BURST` (ten or more transactions within 10 minutes), and `RAPID_MOVEMENT` (large amount in followed by similar amount out within 15 minutes). Because AWS Lambda invocations are ephemeral and cannot retain state between executions, the system uses Amazon DynamoDB as an external state store. For each customer, a rolling window of recent transactions is persisted in DynamoDB and refreshed on every invocation. DynamoDB's native Time-To-Live (TTL) feature automatically purges entries older than two hours, eliminating the need for scheduled cleanup. To minimise per-record latency, the consumer Lambda groups Kinesis records by customer before issuing DynamoDB reads, performing one round-trip per unique customer rather than one per transaction.

Data Quality and the Cleaned-View Layer addresses a common pitfall in data lake design: querying raw, untyped data directly. CSV files store everything as strings, including booleans and timestamps. The project introduces five Athena views (`customers_clean`, `accounts_clean`, `transactions_clean`, `alerts_clean`, `alerts_enriched`) that cast types, filter invalid rows, deduplicate slowly-changing customer snapshots to the latest version, and pre-join alerts with customer

profiles. Downstream consumers query the views, never the raw tables, ensuring consistent data quality.

Slowly-Changing Dimensions (SCD) describes reference data that changes over time. Joining alerts directly to all customer snapshots produces inflated counts and orphan records (alerts whose customers no longer exist in the latest snapshot). The cleaned view layer resolves this by exposing only the most recent customer snapshot.

False Positives and Detection Accuracy are central concerns in any AML system. The project employs a layered scoring model where each of the fourteen rules contributes a 0–100 severity score; an alert fires only when the threshold is met, and multiple rules can fire on the same transaction, producing aggregate evidence. This allows compliance analysts to triage by severity rather than reacting to every single flag.

Security and Compliance is enforced through AWS Identity and Access Management. Each Lambda function operates under its own IAM role with the minimum permissions required (read S3, write Kinesis, etc.). All data in S3 is encrypted at rest using server-side encryption. The dashboard's API access pattern (frontend → FastAPI → Athena) intentionally never exposes AWS credentials to the browser.

3. Related Work

3.1. Existing Systems

Current Anti-Money-Laundering (AML) and fraud detection systems generally fall into three categories: legacy batch-processing platforms, synchronous real-time APIs, and centralised machine learning models. Each of these existing paradigms presents distinct limitations when deployed at enterprise scale.

Legacy Batch and Rule-Based Systems. Traditional compliance platforms such as Actimize, Fircosoft, and FICO Falcon primarily employ batch-oriented architectures that process transactions in overnight cycles. While these legacy systems can achieve high throughput by evaluating millions of transactions per batch, they inherently introduce regulatory exposure by delaying the screening process. Furthermore, these systems historically rely on rigid, static rules that struggle to adapt to evolving criminal tactics. This lack of adaptability results in an overwhelming operational burden, as legacy rule-based systems frequently generate false-positive rates exceeding 95 percent, requiring extensive and costly manual reviews [1].

Synchronous Real-Time Platforms. To address the latency of batch processing, recent commercial systems like Pelican and ComplyAdvantage have introduced real-time capabilities focused on API-based synchronous screening for individual transactions. However, these synchronous architectures struggle to perform effectively at an institutional scale (such as processing millions of daily transactions) due to severe synchronous bottlenecks [1].

Centralised and Cost-Insensitive Machine Learning Models. Many modern systems have transitioned to machine learning (such as Random Forest or Support Vector Machines) to identify complex fraudulent patterns. However, traditional centralised machine learning approaches pose significant data privacy and confidentiality risks by relying on the mass collection and storage of sensitive client information [2]. Furthermore, these classification systems are often evaluated using standard binary measures (such as misclassification error or ROC) that incorrectly assume all classification errors carry the same financial cost. This cost-insensitive approach is unsuitable for financial fraud detection, where failing to detect a fraudulent transaction (a false negative) carries a significantly higher financial impact than a false positive. Centralised machine learning models often fail to identify atypical or emerging fraudulent acts, leading to substantial financial losses for banking institutions [3].

3.2. Academic Studies

Academic research has extensively explored both the infrastructural and algorithmic challenges of real-time AML and fraud detection. Recent studies emphasise that solving the AML problem

requires a dual approach: optimising the data ingestion pipeline for sub-second latency and advancing detection logic to handle severe data imbalance and financial cost dynamics.

Stream Processing Performance in Data Engineering. On the infrastructural front, academic evaluations of distributed data ingestion strategies highlight the trade-offs between different stream processing engines. A comparative performance study of Apache Kafka integrated with Apache Spark Structured Streaming versus Apache Flink reveals distinct operational advantages. The research demonstrates that Kafka-Flink pipelines achieve superior low-latency performance (median latencies of 45 ms) under high-throughput, stateful workloads, such as continuous financial transactions. In contrast, Kafka-Spark Structured Streaming offers better micro-batching efficiency and fault resilience, making it suitable for hybrid batch-stream pipelines [4]. While both options are powerful, they require self-managed cluster operations. The proposed project demonstrates that for institutional volumes that fit within the AWS managed-service envelope, Amazon Kinesis Data Streams combined with AWS Lambda offers comparable end-to-end latency without the operational complexity of cluster management.

Cost-Sensitive Machine Learning. From an algorithmic perspective, researchers argue that traditional classification metrics (like raw accuracy) are deeply flawed for financial fraud detection. Bahnsen et al. note that credit card fraud is fundamentally an example-dependent and cost-sensitive classification problem. They propose an ensemble method utilising Stacking Cost-Sensitive Decision Trees (CSDT) and Cost-Sensitive Logistic Regression (CSLR). Their core argument is that the financial impact of a false negative (failing to detect a fraudulent transaction) is directly proportional to the stolen amount, whereas a false positive only incurs a fixed administrative cost. By embedding these real-world financial costs directly into the model's training and splitting criteria, the cost-sensitive approach optimises for actual financial savings rather than standard predictive accuracy [3]. The proposed system adopts a related principle in its layered scoring model: each detection rule contributes a 0–100 score weighted by typical financial impact, allowing analysts to triage by severity rather than treating all alerts uniformly.

Advanced Deep Learning and Data Privacy. To address the challenges of complex temporal patterns and extreme class imbalance in transactional data, recent academic studies have turned to advanced neural network architectures. Abbassi et al. proposed a hybrid architecture fusing Variational Autoencoders (VAE) and Quantum Long Short-Term Memory (QLSTM) networks. The VAE encodes latent features and handles data sparsity without naive oversampling, while the QLSTM leverages quantum-inspired mechanisms to capture subtle, evolving temporal fraud behaviours. Furthermore, the architecture is deployed within a Federated Learning (FL) framework, allowing multiple banking institutions to collaboratively train anomaly detection models locally, sharing only model weights rather than sensitive raw data [2]. While such advanced models exceed the scope of an undergraduate prototype, the proposed system's clean separation between rule-based detection and the optional aml-system/ml/ extension folder demonstrates that future ML integration is architecturally supported.

3.3. Comparative Analysis

To justify the architectural and algorithmic design of the proposed AML system, it is essential to compare it against both legacy industry standards and state-of-the-art academic frameworks. The comparative analysis focuses on key performance dimensions: data ingestion strategy, latency, detection methodology, scalability, and operational complexity.

Legacy banking systems, which largely depend on overnight batch processing and static rule-based engines, are highly susceptible to regulatory exposure windows and struggle with false-positive rates that often exceed 90 percent. While commercial synchronous API platforms attempt to solve the latency issue, they fail to scale horizontally when subjected to the high throughput of enterprise-level financial transactions [1].

In contrast, recent academic frameworks such as the ComplianceStream architecture and Federated VAE-QLSTM models demonstrate that combining distributed event streaming (via Apache Kafka and Flink) with advanced cost-sensitive machine learning drastically reduces false positives while achieving sub-second processing latency. However, these academic models require self-managed cluster operations and complex deployment strategies that are infeasible for smaller institutions or academic prototypes [2].

The proposed system bridges the gap between theoretical academic benchmarks and practical industry application by utilising a fully serverless, cloud-native architecture. By replacing self-managed open-source streaming clusters (Kafka + Spark) with AWS managed primitives (Kinesis Data Streams + Lambda + Athena), the project achieves the event-driven processing pattern recommended by academic literature while eliminating cluster management entirely. This results in a system that idles at near-zero cost, scales automatically with traffic, and remains within the operational reach of a single developer or a small compliance team.

Feature / Dimension	Legacy AML Systems	Academic Frameworks	Proposed Cloud-Native System
Data Processing	Overnight Batch Processing	Distributed Stream Processing	Event-driven serverless streaming
Ingestion Technology	Mainframe / ETL Scripts	Apache Kafka, Apache Flink	Amazon Kinesis Data Streams
Compute Layer	Self-hosted servers	Self-managed Spark / Flink clusters	AWS Lambda

Storage	Relational databases	HDFS / object storage	Amazon S3
Detection Logic	Static Rule-Based Engines	Complex Neural Nets / Ensembles	14 layered rule patterns with severity scoring
Stateful Memory	Centralized RDBMS, batch-loaded	In-cluster Spark/Flink state	Amazon DynamoDB with TTL (managed, schemaless)
Query Layer	Batch reports	Custom dashboards	Amazon Athena (SQL-on-S3) with cleaned views
Latency	8 - 24 Hours	Sub-second (in-memory only)	Sub-minute (8–11s end-to-end)
Scalability	Vertical, limited	Horizontal, manual cluster sizing	Horizontal, automatic, managed
Idle Cost	High (always-on hardware)	High (always-on cluster)	Near zero
Operational Complexity	High (legacy maintenance)	High (cluster ops, tuning, patching)	Low (managed services, no servers)
Suitability for Prototyping	Poor	Moderate	High

Table 2: Comparative Analysis

A common challenge in event-driven AML systems is maintaining state across stateless serverless invocations. Self-managed alternatives (Spark Structured Streaming with checkpoints, Flink with managed state) require ongoing cluster operation. The proposed system addresses this challenge through Amazon DynamoDB, configured with a Time-To-Live attribute that automatically expires stale state entries after two hours. This approach inherits the operational simplicity of managed services while providing the stateful semantics required for time-windowed AML rules.

The comparative analysis confirms that moving from batch to stream processing is no longer optional for AML compliance. By integrating fully managed cloud primitives with a clean, layered detection model, the proposed system overcomes the primary bottlenecks of traditional financial fraud detection while remaining accessible for academic prototyping and small-scale deployment.

4. Software Process Model

4.1. Selected Software Development Model

For the development of this AML system, an Agile software development methodology has been selected, specifically the Scrum framework. Unlike linear development models such as Waterfall, Agile is iterative and incremental, making it well-suited for data engineering projects. Building a serverless, cloud-native architecture involves continuous refinement; as data pipelines are constructed and detection rules are tuned, unexpected challenges such as handling data quality issues, optimising stream processing latency, or resolving slowly-changing dimension joins frequently arise.

Scrum provides the flexibility necessary to address these complexities. By breaking the project into manageable, time-boxed iterations (Sprints), the development process allows for continuous testing, evaluation, and adaptation. This iterative approach ensured that core functionalities, such as the real-time ingestion of transactional data via Amazon Kinesis, were built, tested, and validated before progressing to dependent features such as the cleaned Athena view layer and the React dashboard.

Adopting Scrum for this project enabled a structured yet adaptable workflow, facilitating the steady delivery of a functional Minimum Viable Product (MVP) while accommodating the inherent uncertainties of working with real-time streaming data and a multi-service cloud stack.

4.2. Definition of Scrum Roles

While Scrum is traditionally designed for multi-disciplinary teams, its principles of iterative development, backlog management, and continuous feedback are highly effective for individual academic projects. For this Final Year Project, the standard Scrum roles have been adapted to fit a single-developer environment, incorporating the project supervisor as a key stakeholder.

Product Owner (Shared Role). In a standard Scrum framework, the Product Owner maximises the value of the product and manages the Product Backlog. For this project, this role is functionally shared between the student and the Project Advisor. The advisor acts as the primary stakeholder, defining academic standards, evaluating sprint deliverables, and providing feedback. The student acts as proxy Product Owner by translating the advisor's feedback and project objectives into actionable tasks, defining the Minimum Viable Product, and prioritising the Product Backlog.

Scrum Master (Self-Managed). The Scrum Master ensures the Agile process is followed and removes impediments. As a solo developer, the student assumes this role through strict self-management and time-boxing. This involves maintaining a project tracker, ensuring each

iteration remains focused, and identifying technical blockers early such as AWS configuration limits or data pipeline issues, so they can be researched or discussed during advisor meetings.

Development Team (Solo Engineer). The Development Team consists entirely of the student, who is responsible for the end-to-end technical execution of the project. This encompasses all roles typically found in a data engineering team, including Cloud Architect (designing the AWS infrastructure), Data Engineer (building the Kinesis and Lambda pipelines), Backend Developer (FastAPI service), and Frontend Developer (React dashboard).

4.3. Scrum Process

The Scrum lifecycle consists of specific events (ceremonies) and artifacts designed to maintain momentum, ensure transparency, and allow for continuous adaptation. For this project, the entire Scrum workflow was managed using a combination of GitHub Issues for backlog tracking and a personal markdown log for daily check-ins and sprint retrospectives.

Backlog Management (Artifacts). GitHub Issues served as the Product Backlog. Each issue captured a specific deliverable (an AWS resource to provision, a Lambda function to write, a Glue crawler to configure, an Athena view to test, a React component to build). Issues were labelled by category (infra, data-pipeline, detection, analytics, frontend, documentation) and assigned a priority. This allowed the most critical components (ingestion before detection, detection before dashboard) to be addressed in dependency order.

Sprint Planning. The project was divided into fixed two-week iterations (Sprints). At the start of each Sprint, high-priority issues were pulled from the backlog into a milestone. A clear Sprint Goal was documented in the milestone description (for example, "End-to-end data flow from EventBridge to S3 partitioned alerts") to maintain focus.

Daily Scrum (Adapted). In a team environment, the Daily Scrum is a brief stand-up meeting. For this solo project, it was adapted into a daily commit message convention that summarised what was changed and what was next. Each commit ended with a brief status note, providing a chronological log of progress and immediate blockers.

Sprint Review. At the conclusion of each two-week iteration, a Sprint Review was conducted with the Project Advisor. The deliverables were demonstrated through working AWS resources, Athena query screenshots, and architecture diagrams. This served as the basis for advisor feedback and prioritisation of the next sprint.

Sprint Retrospective. Following the review, a personal retrospective was written. This reflected on the development process by answering three questions: what went well, what took longer than expected, and what could be improved. Key retrospective findings led to architectural pivots,

most notably the decision to adopt Kinesis + Lambda over the originally-planned MSK + EMR stack after evaluating provisioning time and cost.

4.4. Scrum Implementation in the Project

The development of the AML system was executed across six two-week sprints. The original sprint plan included Apache Kafka (Amazon MSK) and Apache Spark (Amazon EMR); however, after the Sprint 1 retrospective and discussion with the advisor, the architecture was revised to adopt Amazon Kinesis Data Streams and AWS Lambda, eliminating the need for cluster provisioning and reducing the project's operational footprint. The final sprint plan reflects this pivot.

Sprint 1: Project Setup, Cloud Foundation, and Architecture Pivot.

- Established the development environment (Python, Node.js, AWS CLI, Git).
- Provisioned the foundational AWS resources: S3 bucket, IAM roles, initial Lambda execution permissions.
- Initial provisioning experiments with Amazon MSK revealed cluster provisioning times of 20+ minutes and a non-trivial monthly idle cost. After advisor consultation, the architecture was revised to use Amazon Kinesis Data Streams (on-demand mode), which provisions in seconds and bills per record.
- Updated the system architecture diagram to reflect the new stack.

Sprint 2: Real-Time Data Ingestion and Generation Layer (revised).

- Implemented the `Entities` Lambda producing 300 customers and 450 accounts per cycle.
- Implemented the `GenerateTransactions` Lambda producing 100 normal transactions plus eight categories of suspicious patterns (structuring, layering, round-trip, shell-company, trade-based, large-rapid, high-velocity, impossible-travel).
- Implemented the `GenerateAlerts` Lambda producing the batch-alerting comparator.
- Provisioned the `aml-pipeline` Step Functions state machine sequencing all three Lambdas in deterministic order: `Entities` → `GenerateTransactions` → `GenerateAlerts`.
- Configured a single `EventBridge` schedule (`AML-pipeline-scheduler`) firing every 1 minute to trigger the state machine.
- **Architectural note:** an earlier draft used two independent schedules (a 5-minute schedule for entities, a 1-minute schedule for the rest). This was consolidated into a single orchestrated workflow during the sprint to (1) guarantee strict ordering between entity refresh and transaction generation, (2) simplify observability by giving every cycle

a single Step Functions execution to inspect, and (3) reduce schedule sprawl in EventBridge.

- Provisioned the Kinesis Data Stream `transactions-data-stream` and integrated `put_records` into the transaction generator.

Sprint 3: Stream Processing and Real-Time Detection.

- Implemented the AML Consumer Lambda subscribing to the Kinesis stream.
- Wrote the fourteen rule-based detection functions covering placement, layering, and integration phases of money laundering.
- Designed the layered scoring model where each fired rule contributes a 0–100 score.
- Configured the consumer to write JSONL alerts to `s3://.../alerts/realtime/dt=YYYY-MM-DD/`.
- Validated end-to-end flow: a generated suspicious transaction produced an alert in S3 within seconds.
- A key architectural decision in this sprint was provisioning Amazon DynamoDB as a stateful memory layer for the Kinesis consumer. Three of the fourteen AML rules (`STRUCTURING_WINDOW`, `VELOCITY_BURST`, `RAPID_MOVEMENT`) require temporal context that cannot be derived from a single transaction. The consumer Lambda was designed to group records by customer before reading state, minimising round-trip count. DynamoDB's native TTL feature was configured at the attribute level, automatically expiring stale state entries after two hours without requiring scheduled cleanup logic.

Sprint 4: Data Lake Cataloguing and Cleaned Views.

- Provisioned AWS Glue Database `aml_db` and four Glue Crawlers covering customers, accounts, transactions, and real-time alerts.
- Migrated all S3 writes to Hive-style partitioning (`dt=YYYY-MM-DD/`) by updating the upload helpers in each Lambda.
- Authored five Athena views: `customers_clean`, `accounts_clean`, `transactions_clean`, `alerts_clean`, `alerts_enriched`. These handle type casting, future-date filtering, slowly-changing-dimension deduplication, and pre-joined alert enrichment.
- Validated data quality through dedicated Athena queries: orphan-alert detection, duplicate-customer reconciliation, and PEP coverage.

Sprint 5: Backend Service and API Design.

- Implemented a FastAPI backend with six REST endpoints: `/api/health`, `/api/kpis`, `/api/transactions/flagged`, `/api/alerts/live`, `/api/entities/top-risk`, `/api/alerts/breakdown`, `/api/geo/high-risk`.

- Each endpoint composes parameterised SQL against the cleaned Athena views and returns JSON shaped to match the dashboard's component contracts.
- Configured CORS and .env-driven configuration for AWS region, Athena database, and S3 staging directory.

Sprint 6: Frontend Integration, MVP Finalisation, and Testing.

- Connected the existing React/TypeScript dashboard (SENTINEL) to the FastAPI backend.
- Replaced the previously hard-coded mock data hook (useLiveSimulation) with useLiveData, which polls the six backend endpoints in parallel every 30 seconds.
- Implemented multi-view navigation (Overview, Alerts, Transactions, Entities) with placeholder pages for future modules.
- Performed end-to-end testing across the full pipeline (EventBridge → Lambda → Kinesis → Lambda → S3 → Glue → Athena → FastAPI → React).
- Documented data quality findings, AML pattern coverage, and final system metrics.
- Finalised academic documentation, project README, and component-level READMEs.

5. Requirements Analysis

5.1. Stakeholders

The requirements analysis phase establishes a clear understanding of what the AML system must achieve to be considered successful. This chapter defines the individuals and entities invested in the system (Stakeholders), the specific features the system must provide (Functional Requirements), and the performance and architectural standards it must meet (Non-Functional Requirements).

5.2. User Requirements

Stakeholders for this project can be divided into two categories: the Academic Stakeholders involved directly in the project's lifecycle, and the Theoretical (Domain) Stakeholders representing the intended users of the system in a real-world financial environment.

5.2.1. Academic Stakeholders (Directly Involved):

Project Developer (Moavia Mahmood):

The student and sole engineer responsible for the end-to-end design, implementation, testing, and documentation of the AML system. Responsibilities include cloud architecture provisioning, data pipeline engineering, and project management.

Project Advisor (Asst. Prof. Dr. Yücel Tekin):

The primary academic stakeholder and functional Product Owner. The advisor provides technical guidance, ensures the project maintains academic rigor, evaluates Sprint deliverables, and assesses the final system against the initial objectives.

Evaluation Jury / Academic Committee:

The faculty members of the Software Engineering department who will review the final project report, attend the system demonstration, and grade the project based on its technical complexity, execution, and academic merit.

5.2.2. Theoretical Stakeholders (System End-Users & Beneficiaries):

Compliance Officers / Fraud Analysts:

The primary intended end-users of the AML platform. In a real-world setting, these individuals would use the system's output (stored in Amazon Athena/DynamoDB) to investigate transactions flagged by the rule-based detection engine. They rely on the system to provide low false-positive rates to avoid wasting investigative resources.

Financial Institutions (Banks / FinTechs):

The organizations deploying the system. Their primary interest is mitigating financial risk, preventing monetary losses, and avoiding heavy regulatory fines caused by exposure windows in legacy batch-processing systems.

Regulatory Authorities:

Organizations (such as local financial intelligence units or international bodies like the FATF) that enforce AML laws. While they do not use the system directly, they mandate the compliance reports that the analytical database component of this system is designed to generate.

5.2.3. Functional Requirements

Functional requirements specify the exact actions and behaviours the AML system must perform to successfully detect and manage suspicious financial activities. These requirements are categorised according to the logical flow of the data engineering pipeline.

Data Ingestion and Simulation

- **FR-1:** The system shall continuously ingest streaming financial transaction data in real time using **Amazon Kinesis Data Streams**.
- **FR-2:** The system shall include three coordinated data-generation Lambda functions, sequenced by AWS Step Functions: **Entities** (refreshes the customer/account dimension snapshot), **GenerateTransactions** (produces synthetic banking transactions and publishes to Kinesis), and **GenerateAlerts** (produces the batch alerting comparator). The state machine is triggered by a single EventBridge schedule firing every minute.
- **FR-3:** The system shall partition all data written to the data lake by date using Hive-style prefixes (dt=YYYY-MM-DD) to enable efficient downstream querying.

Real-Time Processing and Detection

- **FR-4:** The system shall process incoming Kinesis records continuously through an AWS Lambda consumer, eliminating overnight batch windows.
- **FR-5:** The system shall evaluate every incoming transaction against fourteen predefined AML detection rules covering structuring, layering, round-trip transfers, shell-company routing, trade-based money laundering, large-rapid wires, high-velocity bursts, impossible-travel anomalies, and high-risk geographic exposure.
- **FR-6:** Each rule that fires shall produce an alert with a 0–100 severity score, allowing aggregate scoring when multiple rules fire on the same transaction.

- **FR-7:** The system shall classify a transaction as "suspicious" if any of its ground-truth labels or rule evaluations indicate fraud, and shall persist these classifications for downstream querying.

Data Storage and Querying

- **FR-8:** The system shall securely store all raw transactional data in **Amazon S3** as date-partitioned CSV files for backup, audit, and reprocessing purposes.
- **FR-9:** The system shall store all generated alerts in S3 as date-partitioned JSONL files under alerts/realtime/dt=....
- **FR-10:** The system shall expose a cleaned analytical layer through **Amazon Athena views** (customers_clean, accounts_clean, transactions_clean, alerts_clean, alerts_enriched) that handle type casting, deduplication, and pre-joined enrichment.
- **FR-11:** The system shall allow authorised users (compliance officers) to query the analytical layer via SQL through Athena and through the FastAPI service consumed by the React dashboard.

System Monitoring and Security

- **FR-12:** The system shall log all Lambda invocations, errors, and Kinesis consumer lag metrics through **AWS CloudWatch**.
- **FR-13:** The system shall enforce role-based access control via **AWS IAM**, ensuring each Lambda operates under a least-privilege role with only the necessary S3, Kinesis, and CloudWatch permissions.
- **FR-14:** The system shall provide a real-time React/TypeScript dashboard (SENTINEL) presenting live KPIs, alert breakdowns, top suspicious entities, jurisdictional risk profiles, and a streaming alerts feed driven by real Athena queries.
- **FR-15:** The system shall maintain rolling per-customer transaction history in Amazon DynamoDB (aml-customer-state) to enable temporal AML detection rules (STRUCTURING_WINDOW, VELOCITY_BURST, RAPID_MOVEMENT). State entries shall expire automatically after two hours via DynamoDB TTL.

5.2.4. Non-Functional Requirements

Non-functional requirements specify the quality attributes, performance goals, and architectural constraints of the system. Given the high-velocity nature of financial transactions and the sensitivity of banking data, the system adheres to the following standards:

Performance and Latency

- **NFR-1 (Low Latency):** The end-to-end pipeline (transaction generation → Kinesis → Lambda consumer → S3 alert) shall complete within seconds, eliminating the regulatory exposure windows of legacy batch systems.
- **NFR-2 (Throughput):** The Kinesis Data Stream operating in on-demand capacity mode shall scale automatically to absorb traffic bursts without dropping records.

Scalability

- **NFR-3 (Horizontal Scalability):** The processing layer shall scale horizontally and automatically. AWS Lambda provisions concurrent execution environments per shard, requiring no manual cluster sizing.
- **NFR-4 (Storage Scalability):** The Amazon S3 data lake shall accommodate growing volumes of historical data without manual intervention.

Reliability and Fault Tolerance

- **NFR-5 (High Availability):** All AWS managed services used (Kinesis, Lambda, S3, Athena, Glue) operate across multiple Availability Zones by default and target 99.9 percent availability.
- **NFR-6 (Data Resilience):** Kinesis retains records for 24 hours, allowing the consumer Lambda to retry failed batches automatically with no data loss.

Security and Compliance

- **NFR-7 (Access Control):** All inter-service communication is authenticated through AWS IAM, with each component operating under a dedicated least-privilege role.
- **NFR-8 (Data Encryption):** All data at rest in S3 is encrypted using server-side encryption (SSE-S3). All data in transit between services uses TLS by default.

Maintainability and Usability

- **NFR-9 (Modular Architecture):** The codebase is organised into clearly separated components (data-generators/, ingestion/, pipelines/, frontend/, backend/), allowing changes in one layer (for example, adding a new AML rule) without breaking others.
- **NFR-10 (Query Usability):** The Athena layer supports standard SQL, allowing analysts with basic database knowledge to extract insights and generate ad-hoc reports without writing application code.
- **NFR-11 (Observability):** The system provides operational visibility through CloudWatch logs and the live dashboard's KPI tiles, allowing real-time observation of alert volume, detection rate, and pattern distribution.

5.3. System Specifications

The system operates on a decentralised, cloud-native architecture. Specifications are divided into the local development environment used for code authoring and the cloud infrastructure environment used for deployment.

5.3.1. Local Development Environment Specifications

The local environment is utilized for writing code, testing logic via containerization before cloud deployment, and managing the project.

Hardware Requirements. Multi-core processor (Intel Core i5/i7 or Apple Silicon equivalent), minimum 8 GB RAM (16 GB recommended for running the React dev server, the FastAPI backend, and a browser concurrently), and at least 20 GB of free disk space for project files, Node modules, and Python virtual environments.

Software Requirements. Operating system: Windows 11 (with WSL2), macOS, or any modern Linux distribution. Languages and runtimes: Python 3.11+ (Lambdas, FastAPI), Node.js 18+ (frontend build and dev server), and SQL (Athena queries). Development tools: Visual Studio Code (primary IDE), Git (version control), AWS Command Line Interface configured with credentials in region eu-north-1.

5.3.2. Cloud Infrastructure Specifications (AWS)

The production-grade infrastructure relies entirely on AWS managed services. All services are provisioned in region eu-north-1 (Stockholm).

Real-Time Streaming (Amazon Kinesis Data Streams).

- Stream: transactions-data-stream
- Capacity mode: on-demand (auto-scales with traffic)
- Retention: 24 hours
- Partition key: sender_customer (ensures ordering per customer for stateful rules)

Serverless Compute (AWS Lambda).

- Runtime: Python 3.11
- Memory: 512 MB per function
- Timeout: 60 seconds
- Functions: Entities-Generator, GenerateTransactions, GenerateAlerts, aml-kinesis-consumer
- Trigger types: EventBridge (entities), Step Functions (orchestrated), Kinesis (consumer)

Event Scheduling (Amazon EventBridge).

- Schedule: `AML-pipeline-scheduler`
- Cadence: `rate(1 minute)`
- Target: AWS Step Functions state machine `aml-pipeline`
- Permissions: dedicated IAM role with `states:StartExecution` on the target state machine

Workflow Orchestration (AWS Step Functions).

- State machine: `aml-pipeline`
- Type: Standard
- Sequence: `Entities` → `GenerateTransactions` → `GenerateAlerts`
- Failure handling: each step retries up to two times with exponential backoff before failing the execution
- Permissions: `lambda:InvokeFunction` on each of the three target Lambdas

Storage (Amazon S3).

- Bucket: `aml-fyp-stream-bucket-<account-id>-eu-north-1-an`
- Layout: `aml-data/{customers|accounts|transactions|alerts}/dt=YYYY-MM-DD/...`
- Encryption: SSE-S3 (default)
- Lifecycle policies: optional Glacier transition for data older than 90 days

Stateful Storage (Amazon DynamoDB).

- Table: `aml-customer-state`
- Primary key: `customer_id` (string, partition key)
- Attributes: `window` (list of `{ts, amount, tx_id}` entries), `ttl` (epoch seconds)
- Capacity mode: on-demand (pay-per-request)
- TTL attribute: `ttl` (enabled, automatic deletion)
- Traffic pattern: one `GetItem` + one `PutItem` per unique customer per Kinesis batch

Data Catalog (AWS Glue).

- Database: `aml_db`
- Crawlers: `aml-customers-crawler`, `aml-accounts-crawler`, `aml-transactions-crawler`, `aml-alerts-realtime-crawler`
- Schedule: hourly (production) or on-demand (development)

Analytical SQL Layer (Amazon Athena).

- Workgroup: primary
- Query result location: s3://.../athena-results/
- Cleaned views: customers_clean, accounts_clean, transactions_clean, alerts_clean, alerts_enriched

Observability (Amazon CloudWatch).

- Log groups: one per Lambda, default 30-day retention
- Metrics: invocation count, error rate, duration, Kinesis iterator age

5.3.3. Network and Security Specifications

Identity and Access Management (IAM). Each Lambda function operates under a dedicated role with the principle of least privilege:

- **LambdaEntityGeneratorRole:** S3 PutObject on aml-data/customers/* and aml-data/accounts/*
- **LambdaTransactionsRole:** S3 read on entity prefixes, S3 PutObject on transactions prefix, Kinesis PutRecords on transactions-data-stream
- **LambdaConsumerRole:** Kinesis read on the stream, S3 PutObject on alerts/realtime/* Plus dynamodb:GetItem and dynamodb:PutItem on the aml-customer-state table.
- **AMLGlueCrawlerRole:** S3 read on the entire aml-data/ prefix, full Glue Catalog access for crawler operations

Encryption. Server-side encryption (SSE-S3) is enabled by default on all bucket objects. All inter-service communication within AWS uses TLS by default.

Public Access. No AWS resource is publicly accessible. The FastAPI backend runs locally during demonstration and queries Athena server-side; the React frontend never receives AWS credentials.

6. System Modeling

6.1. Context Diagram

The context diagram identifies the system boundary and the external actors that interact with it. The AML pipeline is shown as a single black box, with three categories of external interaction: data sources providing raw events, human users consuming detection results, and AWS-managed services providing the runtime substrate.

Diagram description (build in draw.io):

A central rectangle labelled SENTINEL , Real-Time AML System, with the following external entities arrayed around it:

- **Compliance Analyst** (top-left, actor stick figure) , views the dashboard, drills into entities and alerts, exports CSV reports
- **Synthetic Data Generator** (left, rectangle) , provides simulated transaction streams (in production, this would be replaced by core banking systems)
- **AWS Cloud Services** (bottom, rectangle) , provides the underlying runtime: compute (Lambda), streaming (Kinesis), storage (S3, DynamoDB), and analytics (Athena, Glue)

Arrows:

- Synthetic Data Generator → System: "raw transactions"
- System → Compliance Analyst: "alerts, KPIs, reports"
- System ↔ AWS Cloud Services: "uses"

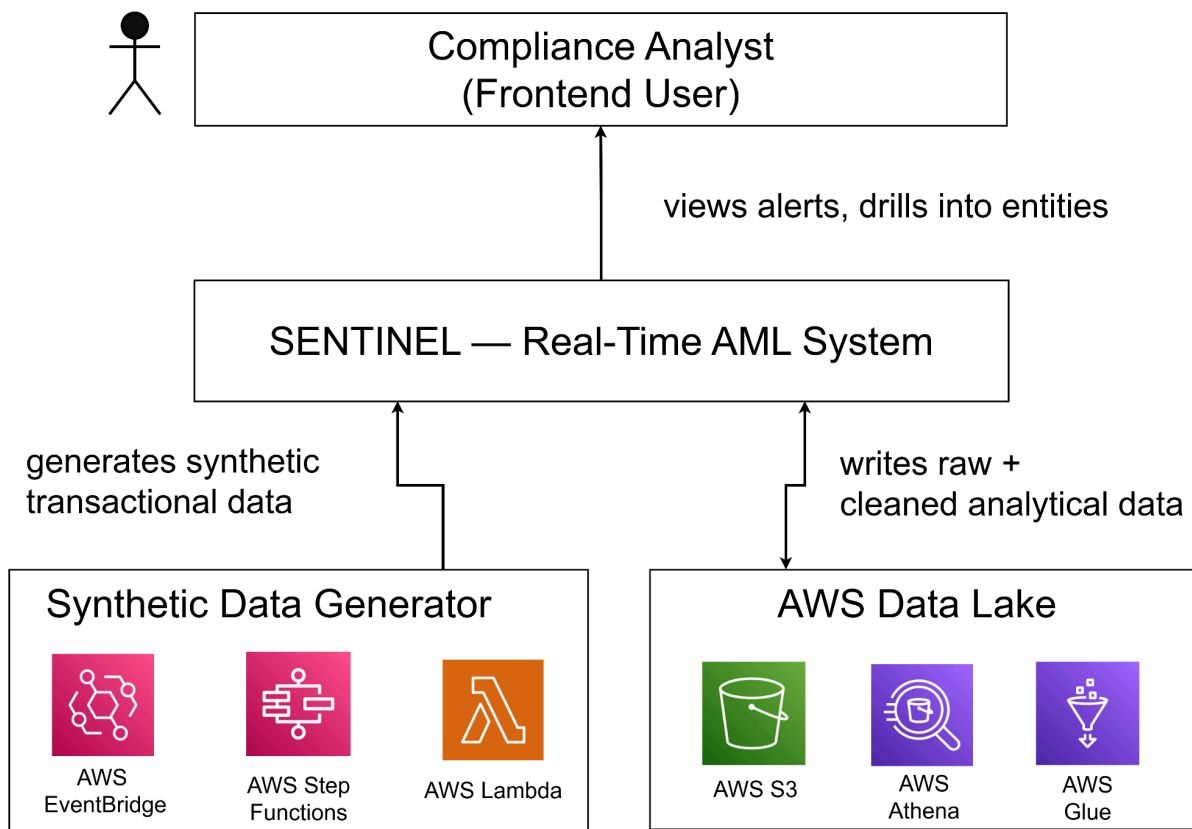


Figure 1: Context Diagram

6.2. Use Case Diagram

The use case diagram captures the functional interactions between actors and the system. Two actors are identified: the **Compliance Analyst** (primary end-user, in the production scenario) and the **System Administrator** (the developer, during operation and maintenance).

Diagram description (build in draw.io):

System boundary: a rectangle labelled AML Detection System.

Actors (stick figures, outside the boundary):

- **Compliance Analyst** (Top)
- **System Administrator** (Bottom)

Use cases (ovals inside the boundary):

Compliance Analyst connects to:

- View Real-Time KPIs
- Inspect Alert Breakdown by AML Pattern
- Drill into Top Suspicious Entities
- Filter and Search Flagged Transactions
- View Geographic Risk Distribution
- Export Transactions as CSV

System Administrator connects to:

- Trigger Manual Data Generation
- Run Glue Crawler on Demand
- Query Data Lake via Athena Console
- Inspect CloudWatch Logs

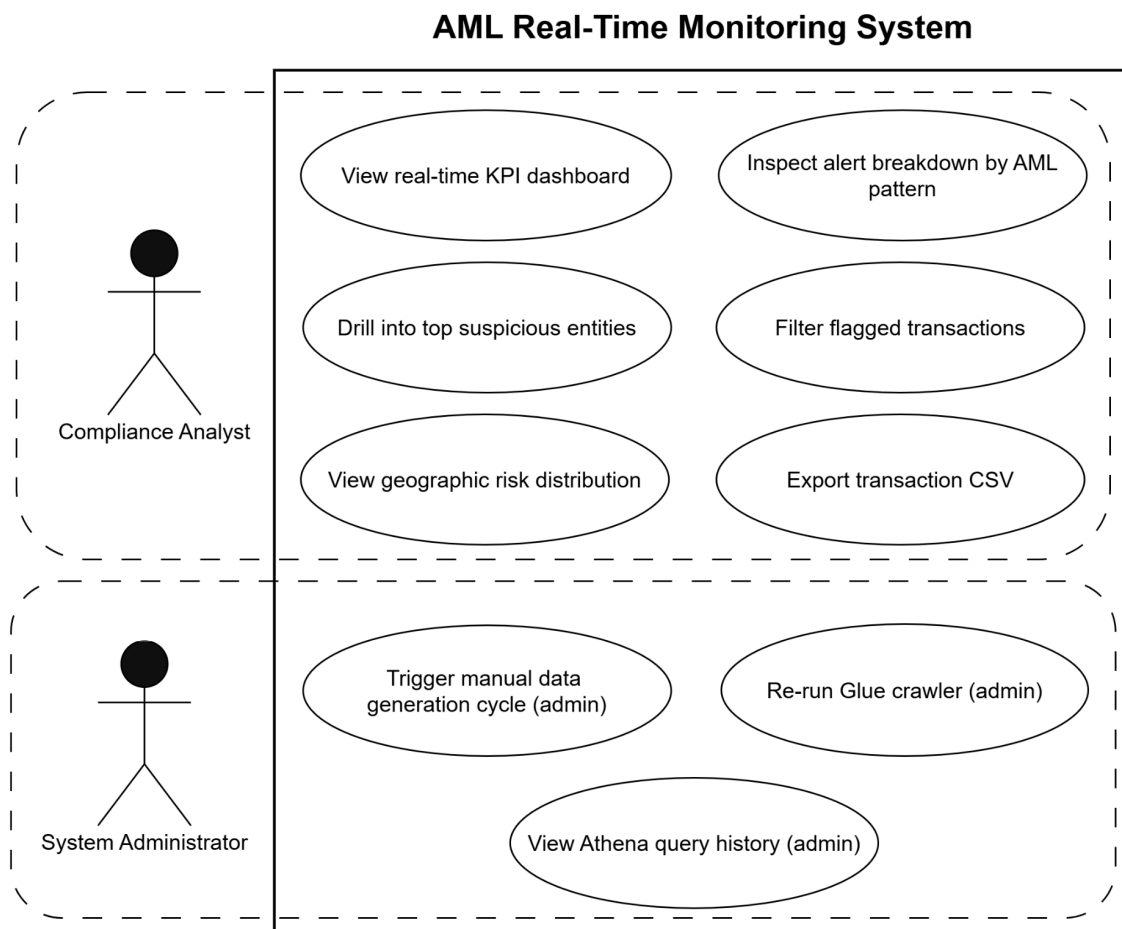


Figure 2: Use Case Diagram

6.3. Use Case Descriptions

The following table summarises the most important use cases:

Use Case	Actor	Description	Trigger
View Real-Time KPIs	Compliance Analyst	Open the SENTINEL dashboard and observe live alert counts, suspicious volume, and pattern distribution. The dashboard auto-refreshes every 30 seconds.	User loads the application URL
Inspect Alert Breakdown	Compliance Analyst	View a horizontal bar chart showing the count of each AML pattern detected. Tap a bar to see severity scores.	User opens the Overview view
Drill into Suspicious Entity	Compliance Analyst	Click on a customer in the "Top Risk Entities" panel. A modal opens showing the customer's transaction history, risk score breakdown, and PEP status.	Click on entity row
Filter Flagged Transactions	Compliance Analyst	In the Transactions Table, apply filters (Critical / High / Medium / Cleared) or search by entity name.	Click filter button
Export CSV	Compliance Analyst	Download the current transaction list as a CSV file for offline review.	Click "Export CSV" button
Trigger Manual Generation	System Administrator	Manually invoke the EventBridge schedule or directly trigger a Lambda for testing.	AWS Console, manual action
Query Data Lake	System Administrator	Open Athena query editor and run ad-hoc SQL against aml_db.alerts_clean for compliance reporting.	AWS Console

Table 3 : Use Case Descriptions

6.4. Sequence Diagrams

The sequence diagram below traces the lifecycle of a single suspicious transaction from generation to dashboard display.

Lifelines (vertical dashed lines, left to right):

1. EventBridge
2. Step Functions
3. GenerateTransactions Lambda
4. S3 (raw)
5. Kinesis Data Stream
6. AML Consumer Lambda
7. DynamoDB (aml-customer-state)
8. S3 (alerts)
9. Athena (via FastAPI)
10. React Dashboard

Messages (arrows between lifelines, from top to bottom):

1. EventBridge → Step Functions: trigger (every 1 min)
2. Step Functions → GenerateTransactions: invoke
3. GenerateTransactions → S3 (raw): read latest customers/accounts
4. S3 (raw) → GenerateTransactions: CSV data
5. GenerateTransactions → S3 (raw): write transactions_*.csv (partitioned)
6. GenerateTransactions → Kinesis: put_records (batch of N)
7. Kinesis → AML Consumer: invoke (batch trigger)
8. AML Consumer → AML Consumer: group records by customer
9. AML Consumer → DynamoDB: GetItem(customer_id) (per unique customer)
10. DynamoDB → AML Consumer: existing window
11. AML Consumer → AML Consumer: prune old + append new + evaluate 14 rules
12. AML Consumer → DynamoDB: PutItem(customer_id, updated_window, ttl)
13. AML Consumer → S3 (alerts): write alerts.jsonl (partitioned)
14. (Asynchronous, on user dashboard load)
15. React Dashboard → FastAPI: GET /api/kpis
16. FastAPI → Athena: SELECT ... FROM alerts_clean
17. Athena → S3 (alerts): scan partitions
18. S3 (alerts) → Athena: JSONL records
19. Athena → FastAPI: result set
20. FastAPI → React Dashboard: JSON response
21. React Dashboard → Compliance Analyst: render updated UI

lifecycle of a single suspicious transaction

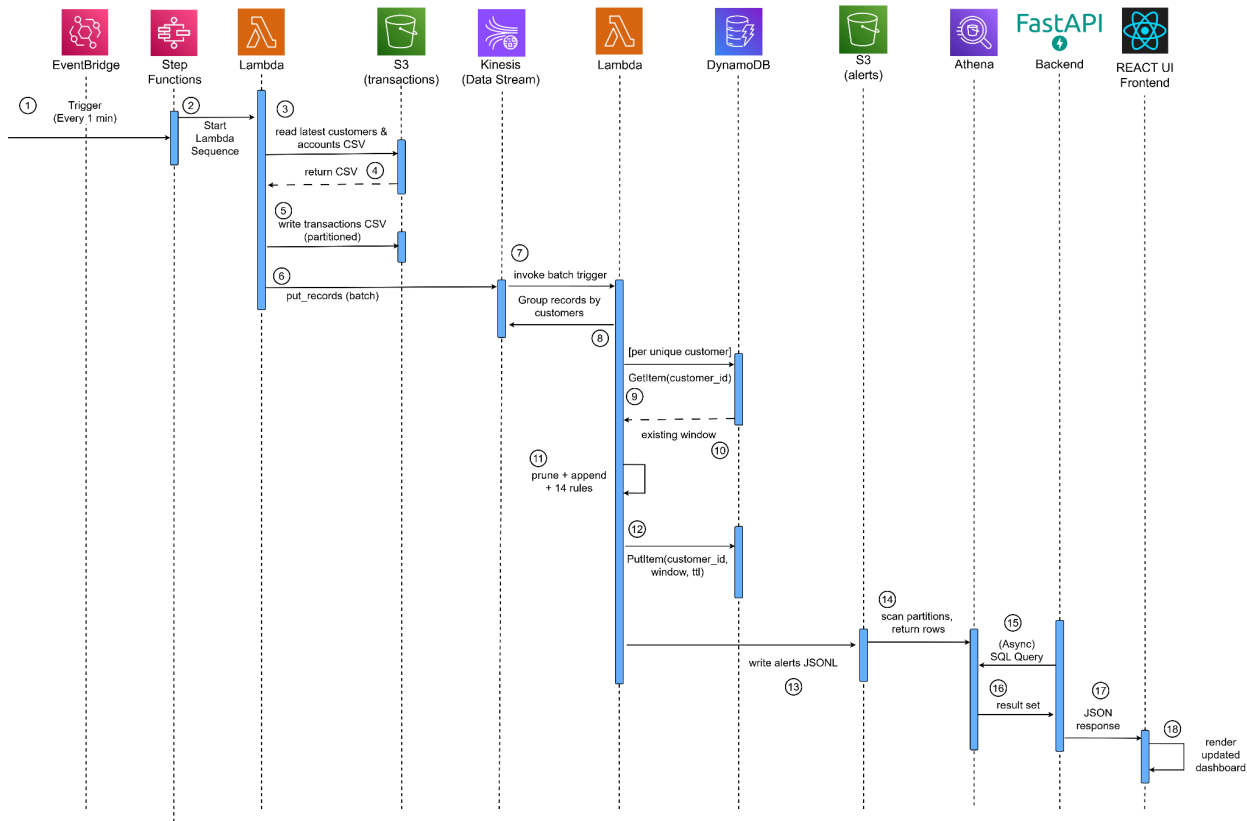


Figure 3: Sequence Diagram

7. System Architecture and Design

7.1. System Architecture

The system follows a **five-layer event-driven architecture** built entirely on AWS managed services. Each layer has a single responsibility, and inter-layer communication is asynchronous wherever possible.

Layer 1 Scheduling. Amazon EventBridge fires one schedule: AML-pipeline-scheduler (every 1 minute). EventBridge schedules are managed and durable.

Layer 2 Data Generation. Three Python 3.11 Lambdas produce synthetic banking data, sequenced by a single AWS Step Functions state machine (**aml-pipeline**). The first Lambda, **Entities**, generates 300 customers and 450 accounts as date-partitioned CSV files in S3, refreshing the slowly-changing customer dimension on every cycle. The second Lambda, **GenerateTransactions**, reads the latest entity snapshots, builds a mix of 100 normal transactions and eight categories of suspicious patterns (structuring, layering, round-trip, shell-company, trade-based, large-rapid, high-velocity, impossible-travel), writes the result to S3 in partitioned CSV form, and publishes each record to Kinesis. The third Lambda, **GenerateAlerts**, produces the batch-alerting comparator that runs in parallel to the real-time detection path. The entire workflow is triggered by a single EventBridge schedule firing every minute, ensuring deterministic ordering between entity refresh, transaction generation, and batch alerting.

Layer 3 Streaming and Detection. Amazon Kinesis Data Streams (transactions-data-stream) acts as the event bus. Records are partitioned by sender_customer to guarantee in-order processing per customer. The AML Consumer Lambda subscribes to the stream and evaluates each transaction against fourteen rules: ten stateless rules (HIGH_VALUE, CROSS_BORDER_HIGH, HIGH_RISK_COUNTRY, SANCTIONED_COUNTRY, plus seven upstream-pattern flags) and three stateful rules (STRUCTURING_WINDOW, VELOCITY_BURST, RAPID_MOVEMENT). Stateful rules read from and write to DynamoDB.

Layer 4 Stateful Memory. Amazon DynamoDB table aml-customer-state holds rolling per-customer transaction windows. The schema uses customer_id as the partition key and stores a list attribute window containing recent transaction timestamps and amounts, plus a ttl attribute that triggers automatic deletion after two hours. The consumer Lambda groups incoming records by customer to perform one read and one write per unique customer per batch.

Layer 5 Analytics and Application. Amazon S3 (data lake), AWS Glue (catalog), Amazon Athena (cleaned views), FastAPI (REST backend), and React/TypeScript (dashboard) form the consumer side of the pipeline. Glue crawlers register Hive-style partitions (dt=YYYY-MM-DD) into the Glue Data Catalog. Athena views layer type casting, deduplication, and pre-joined enrichment over the raw tables. FastAPI exposes seven REST endpoints over HTTPS to the React frontend, which polls every 30 seconds.

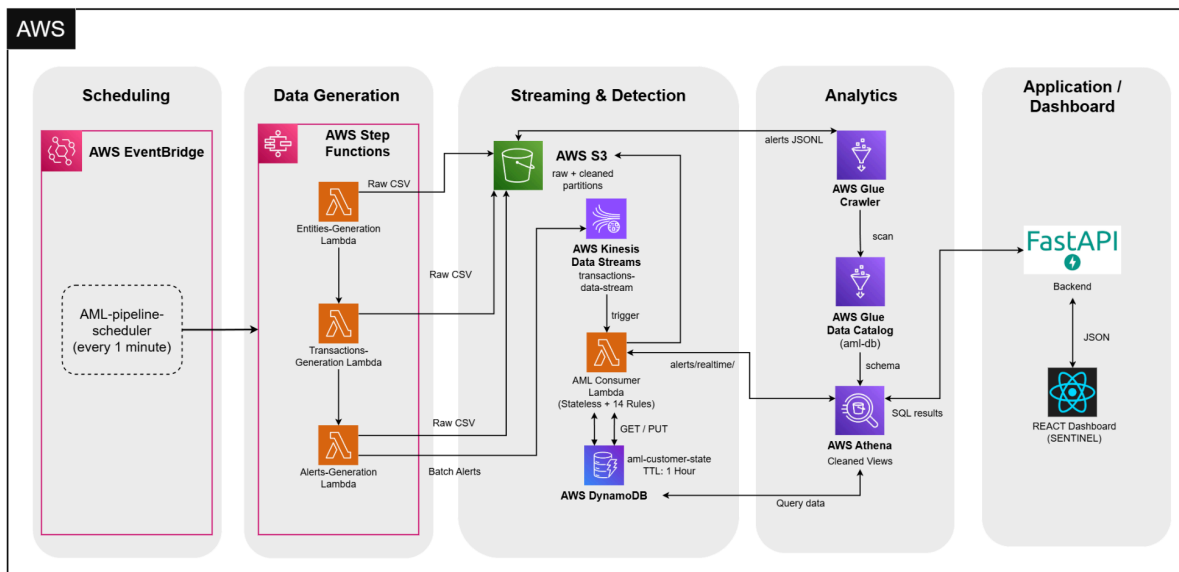


Figure 4: System Architecture

7.2. Structural Models

7.2.1 Data Model (Entity-Relationship Diagram)

The data model is implemented as Python `@dataclass` types in the data-generator Lambdas and matches the schema written to S3 and ingested by Athena. The four primary classes are Customer, Account, Transaction, and Alert.

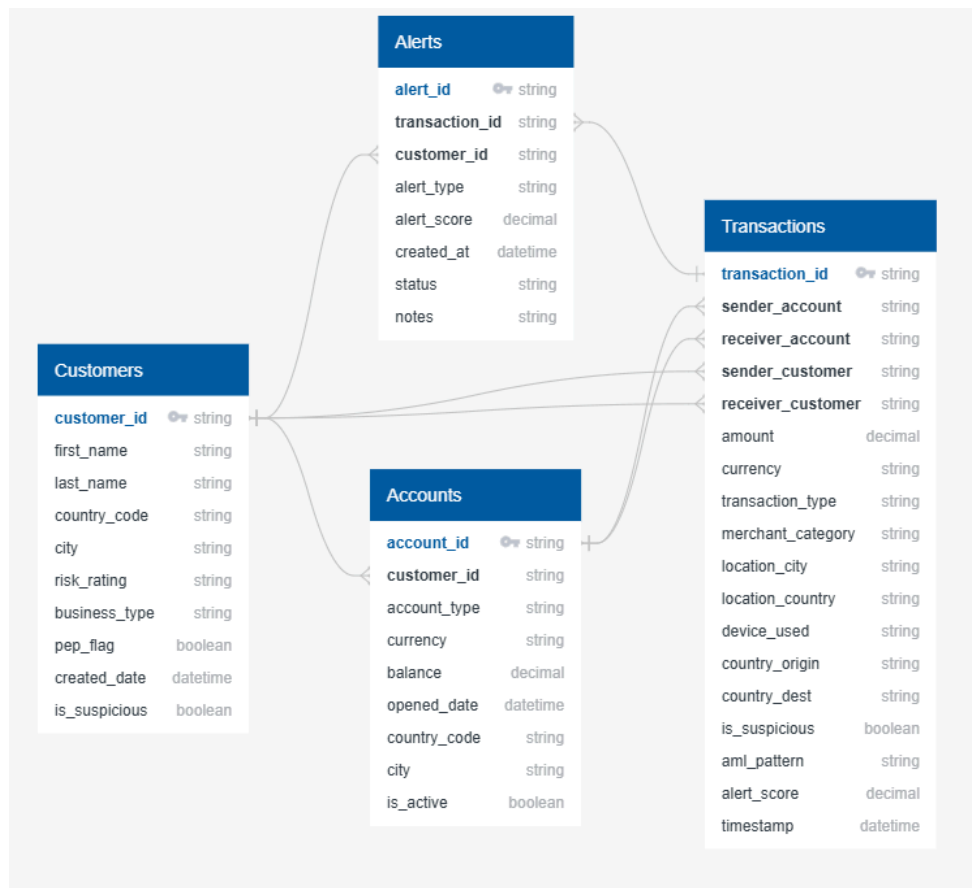


Figure 5: Class Diagram

7.2.2. Inheritance (Generalization)

The current data model does not employ explicit class inheritance, as each entity (Customer, Account, Transaction, Alert) represents a distinct concept with non-overlapping attributes. Inheritance was deliberately avoided in favour of composition, simplifying serialisation to and from CSV and JSON. A future extension could introduce a Stakeholder superclass generalising both Customer and Account, but this would add complexity without clear benefit at the current scope.

7.2.3. Aggregation / Composition

The relationships in the class diagram represent **aggregation** rather than composition. A Customer aggregates one or more Account objects, but accounts are referenced by customer_id foreign key rather than embedded, allowing accounts to outlive an in-memory Customer object as they are written to a separate S3 prefix and a separate Glue table. Similarly, Alert aggregates a Transaction reference by transaction_id. This loose coupling matches the data-lake pattern where each entity type is stored independently and joined on demand at query time through Athena's alerts_enriched view.

7.3. Behavioral Models

7.3.1. Event-Driven Model

The system is fundamentally event-driven: no component polls another. Each layer reacts to upstream events and emits its own. A single transaction's journey involves at least seven asynchronous events:

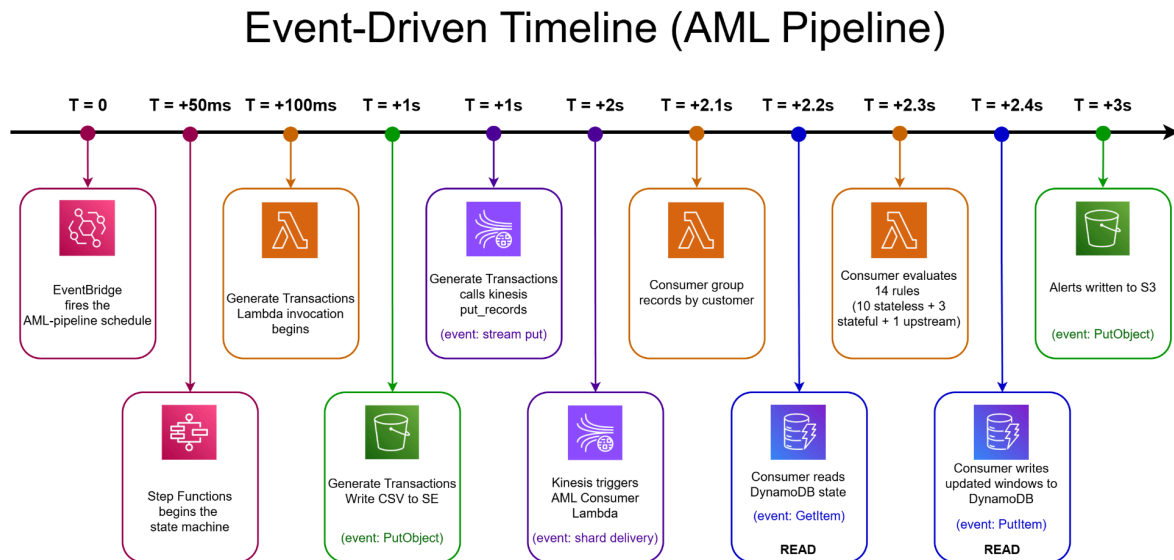


Figure 6: Event-Driven Model

The benefit of this model is full decoupling: any layer can be scaled, replaced, or temporarily unavailable without breaking upstream producers. Kinesis retains records for 24 hours, so a consumer outage does not result in data loss.

7.3.2. State Diagram

The Alert entity follows a four-state lifecycle, modelling the typical compliance triage workflow:

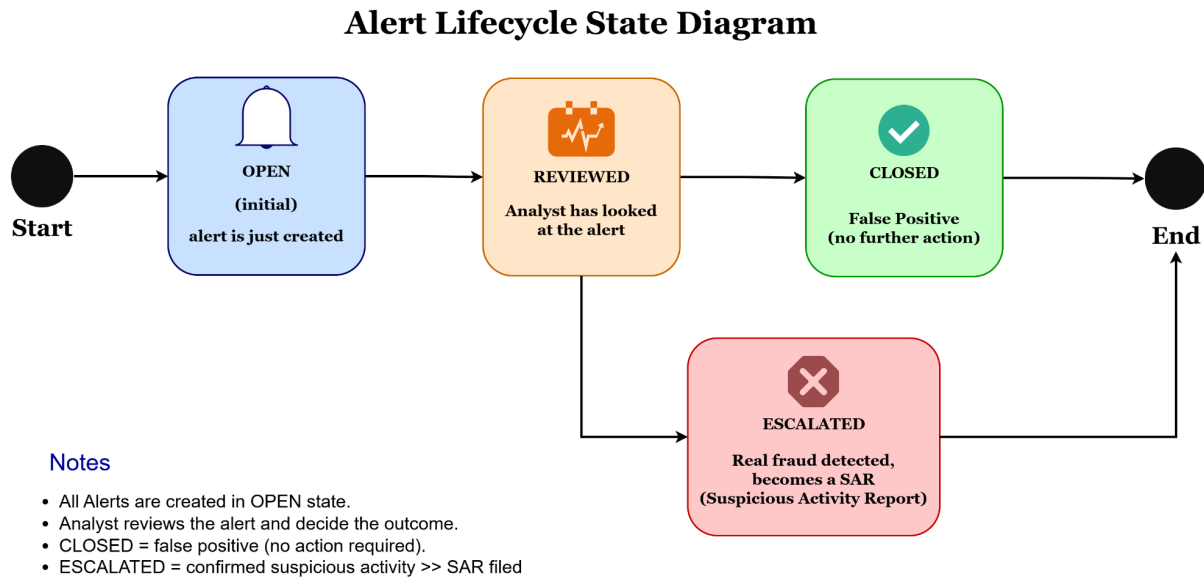


Figure 7: State Diagram

Initial state: OPEN (assigned by the AML Consumer when an alert is created). An analyst either marks the alert as REVIEWED (recorded but no action), CLOSED (false positive, dismissed), or ESCALATED (genuine fraud, the alert becomes the basis for a Suspicious Activity Report filing).

8. System Implementation

8.1. Development Environment

The project was developed on a single Windows 11 workstation with the following tooling:

- **Python 3.11** for all Lambda functions and the FastAPI backend
- **Node.js 18 (LTS)** with **Vite** for the React frontend bundler
- **TypeScript 5** for the dashboard's type-safety layer
- **AWS CLI v2** for managing cloud resources from the command line
- **Visual Studio Code** as the primary IDE, with extensions for Python (Pylance), React (ES7+ snippets), and AWS Toolkit
- **Git** with GitHub for version control and collaboration

The local Python environments are isolated per service: the Lambdas use one virtual environment in `aml-system/data-generators/`, the consumer uses another in `aml-system/ingestion/`, and the FastAPI backend uses its own in `backend/`. The frontend uses npm workspace isolation.

For testing AWS interactions locally, the boto3 library was used directly (without LocalStack or moto), relying instead on rapid Lambda redeployment and CloudWatch logs as the feedback loop.

8.2. System Components

The system consists of six deployable units:

1. Data Generator Lambdas (`aml-system/data-generators/`). Three Python files: `entity-generator_LAMBDA_FUNCTION.py`, `transactions-generations_LAMBDA_FUNCTION.py`, `alert-generation_LAMBDA_FUNCTION.py`. Each is independently deployed to AWS Lambda with environment variables `BUCKET_NAME`, `S3_PREFIX`, and (for the transaction generator) `KINESIS_STREAM_NAME`.

2. AML Consumer Lambda (`aml-system/ingestion/aml-kinesis-consumer_LAMBDA.py`). Subscribes to the Kinesis stream, reads and writes DynamoDB state, and emits JSONL alerts to S3. Triggered automatically by Kinesis with a batch size of 100 records.

4. Step Functions State Machine (`aml-pipeline`). Sequences the three data-generation Lambdas in deterministic order — `Entities`, then `GenerateTransactions`, then

GenerateAlerts — on a 1-minute schedule triggered by EventBridge. Provides centralised observability: every minute produces exactly one state-machine execution visible in the AWS Console, with per-step duration, input/output, and retry history.

4. DynamoDB Table (aml-customer-state). Provisioned through the AWS Console with customer_id as the partition key, on-demand capacity, and TTL enabled on the ttl attribute. The table is referenced by the consumer Lambda via the STATE_TABLE environment variable. No application code is deployed to DynamoDB itself, it is purely a managed key-value store.

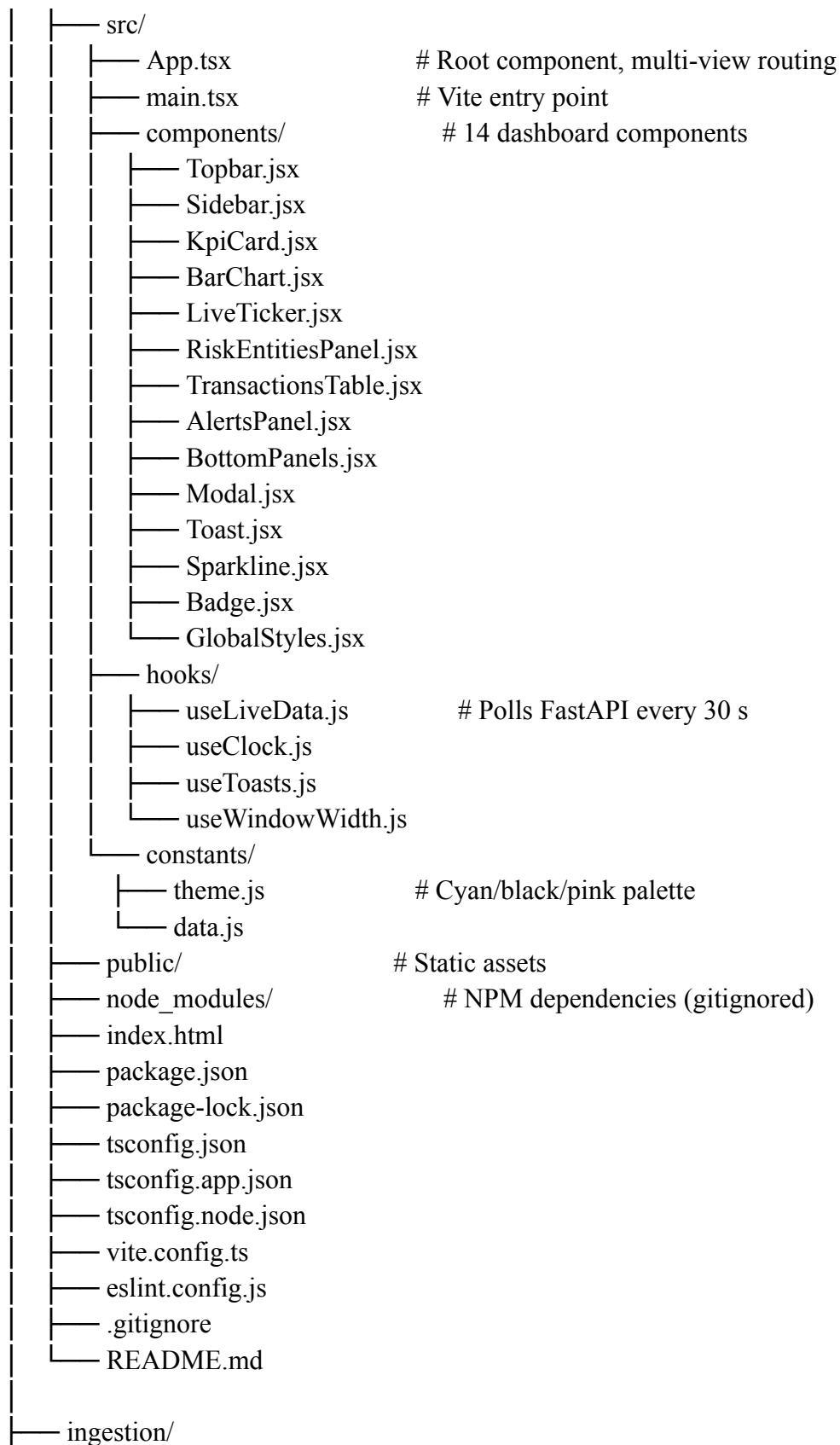
5. Glue Crawlers and Athena Views (aml-system/sql/views.sql). Defined as SQL DDL, applied through the Athena query editor. Five views: customers_clean, accounts_clean, transactions_clean, alerts_clean, alerts_enriched.

6. FastAPI Backend (backend/main.py). A single 200-line Python file implementing seven REST endpoints. Runs locally during demos via uvicorn main:app --reload --port 8000.

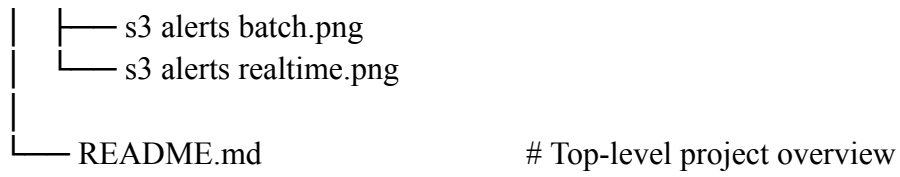
7. React Dashboard (aml-system/frontend/). A Vite-bundled TypeScript SPA. Components in src/components/, hooks in src/hooks/, theme in src/constants/. Built with npm run build to a static dist/ folder.

8.3. Code Structure

```
aml-system/
├── backend/
│   ├── main.py           # FastAPI service (7 endpoints)
│   ├── .env              # Local config (Athena DB, S3 staging)
│   ├── .gitignore
│   └── venv/              # Python virtual environment (gitignored)
├── data-generators/
│   ├── entity-generator_LAMBDA_FUNCTION.py # Customer + account generator (every 5
min)
│   └── transactions-generations_LAMBDA_FUNCTION.py # Transaction generator +
Kinesis publisher
├── alert-generation_LAMBDA_FUNCTION.py # Batch alert comparator
├── output/                # Local test artefacts (gitignored)
├── .env
├── .gitignore
├── README.md
└── frontend/
```



- aml-kinesis-consumer_LAMBDA.py # Real-time AML detection (14 rules)
- .gitignore
- README.md
- pipelines/ # Step Functions definitions
 - step-functions.json
 - README.md
- ml/ # Reserved for future ML scoring
 - figures/
 - evaluation.ipynb
 - requirements.txt
 - .gitignore
- docs/
 - articles/ # Reference papers and reading
 - ANTI-MONEY LAUNDERING DETECTION SYSTEM.pdf # FYP report
 - README.md
 - Context Diagram.drawio + .png
 - Use Case Diagram.drawio + .png
 - Sequence Diagram.drawio + .png
 - System Architecture Diagram.drawio + .png
 - State Diagram.drawio + .png
 - Event-Driven Timeline.drawio + .png
 - QuickDBD-export.png # ER diagram
 - Streaming.drawio + .png # Original architecture sketch
 - data_generation.drawio + .png
 - aws athena query.png # Section 9 / 10 screenshots
 - AWS cloudWatch.png
 - AWS GLUE tables.png
 - aws glue.png
 - AWS Kinesis.png
 - DynamoDB.png
 - S3.png
 - s3 customers.png
 - s3 accounts.png
 - s3 transactions.png
 - s3 alerts.png



The codebase totals approximately 2,500 lines: 1,200 lines of Python (Lambdas + backend), 1,000 lines of TypeScript/JSX (frontend), 200 lines of SQL (Athena views), and 100 lines of configuration (JSON, YAML, env files).

9. MVP Development

9.1. MVP Definition

The Minimum Viable Product was defined at the start of Sprint 1 as the smallest functional system that demonstrates **end-to-end data flow with real AML detection**. Specifically, the MVP must satisfy these criteria:

1. Generate synthetic banking data on a schedule, automatically.
2. Stream transactions through Amazon Kinesis in real time.
3. Apply at least ten distinct AML detection rules to the stream.
4. Persist raw and processed data in a partitioned S3 data lake.
5. Catalog the data for SQL querying via Athena.
6. Provide a live dashboard showing real KPIs and detected alerts.

Stretch goals identified for post-MVP iterations: stateful detection rules using DynamoDB, multi-page navigation in the dashboard, data quality cleaning views, and partition projection optimization.

9.2. Implemented Features

By project completion, the MVP scope plus all stretch goals were delivered:

- **Automated data generation** on EventBridge schedules (entities every 5 minutes, transactions every 1 minute)
- **Real-time streaming** via Amazon Kinesis Data Streams with on-demand capacity
- **Fourteen AML detection rules** covering placement, layering, and integration phases (10 stateless + 3 stateful + 1 upstream-tag rule)
- **Stateful temporal detection** via Amazon DynamoDB with TTL-managed state windows
- **Partitioned data lake** on S3 with Hive-style date partitioning
- **Glue catalog** with four crawlers and a managed schema registry
- **Five Athena cleaned views** handling type casting, deduplication, and pre-joined enrichment
- **FastAPI backend** with seven REST endpoints
- **React/TypeScript dashboard** (SENTINEL) with multi-page navigation, live ticker, KPI tiles, alert breakdown chart, top-entities panel, transactions table, geographic risk panel, and live alerts feed
- **Auto-refresh** every 30 seconds against the real Athena-backed data
- **Quantified data quality findings** (orphan alerts, dropped invalid rows, PEP coverage) documented in the report

9.3. System Interfaces

The MVP exposes three interfaces:

1. **The SENTINEL Dashboard** (web UI). The primary user interface. Loads at <http://localhost:5173> during development. Provides the Overview, Alerts, Transactions, and Entities views with real-time data refresh.

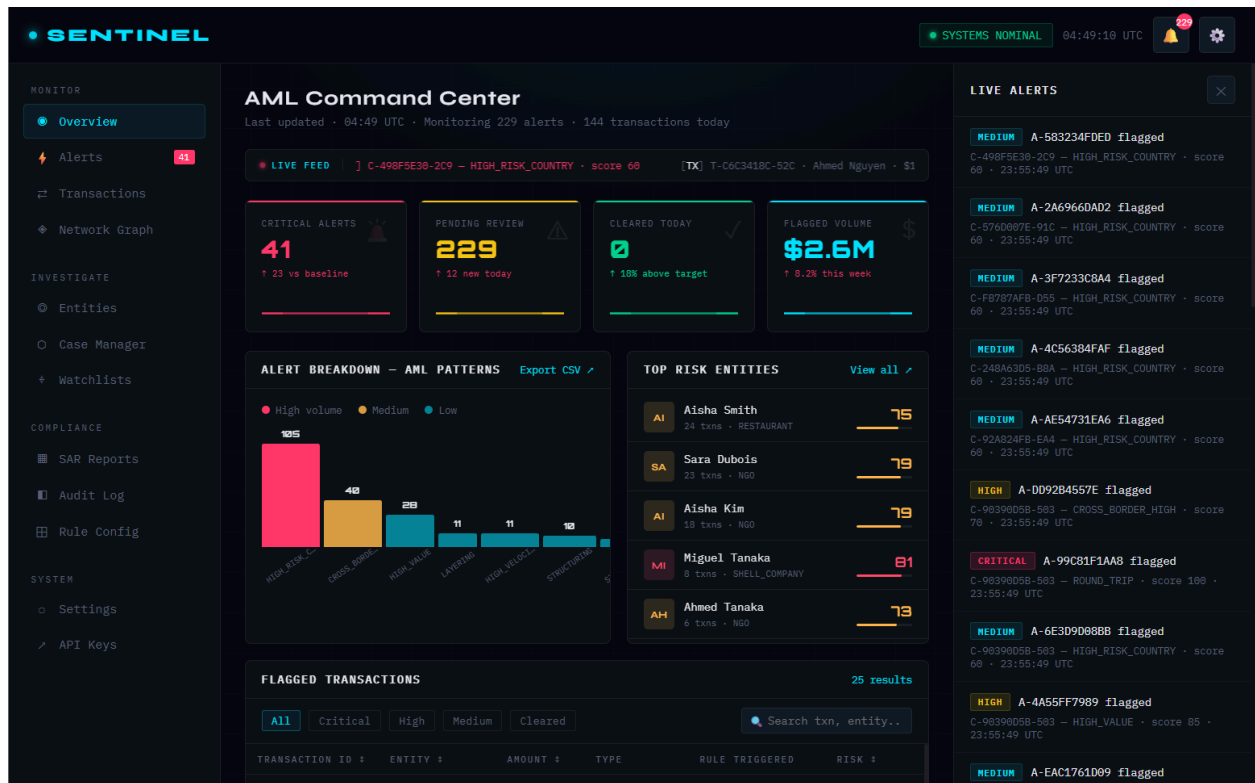


Figure 8: Overview

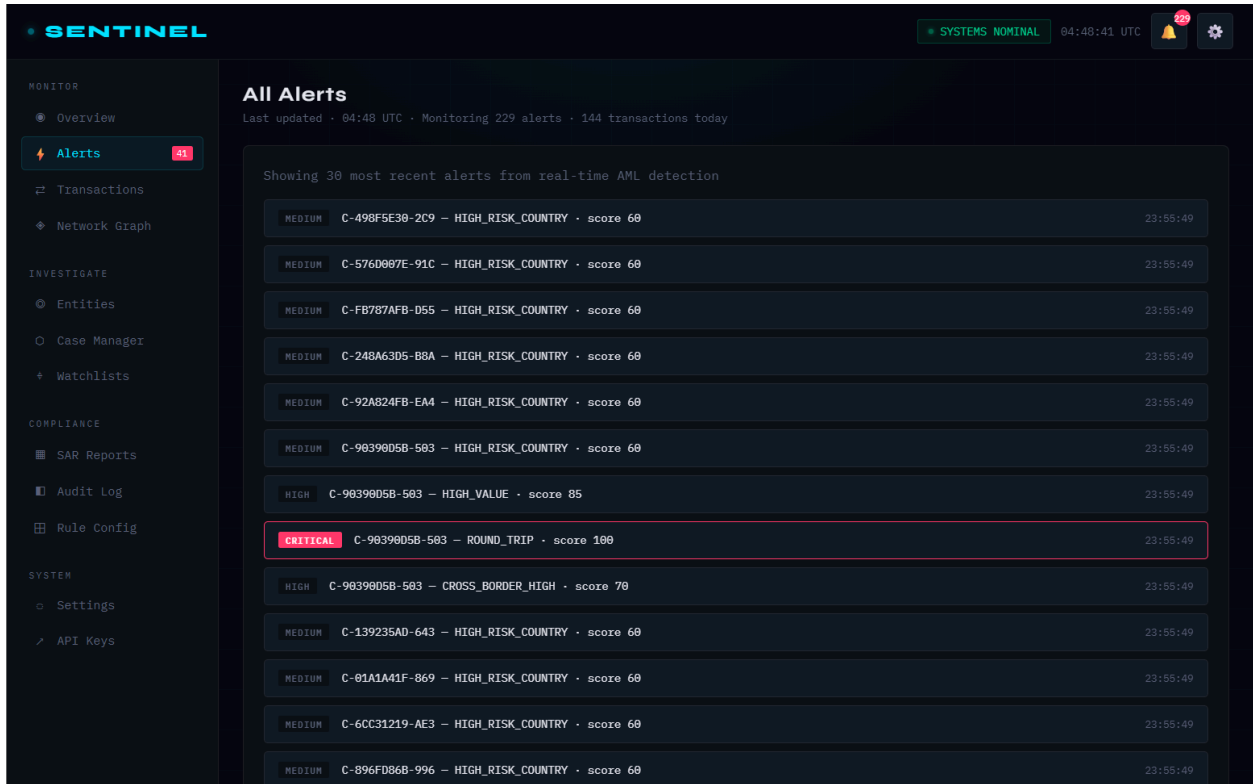


Figure 9: Alerts

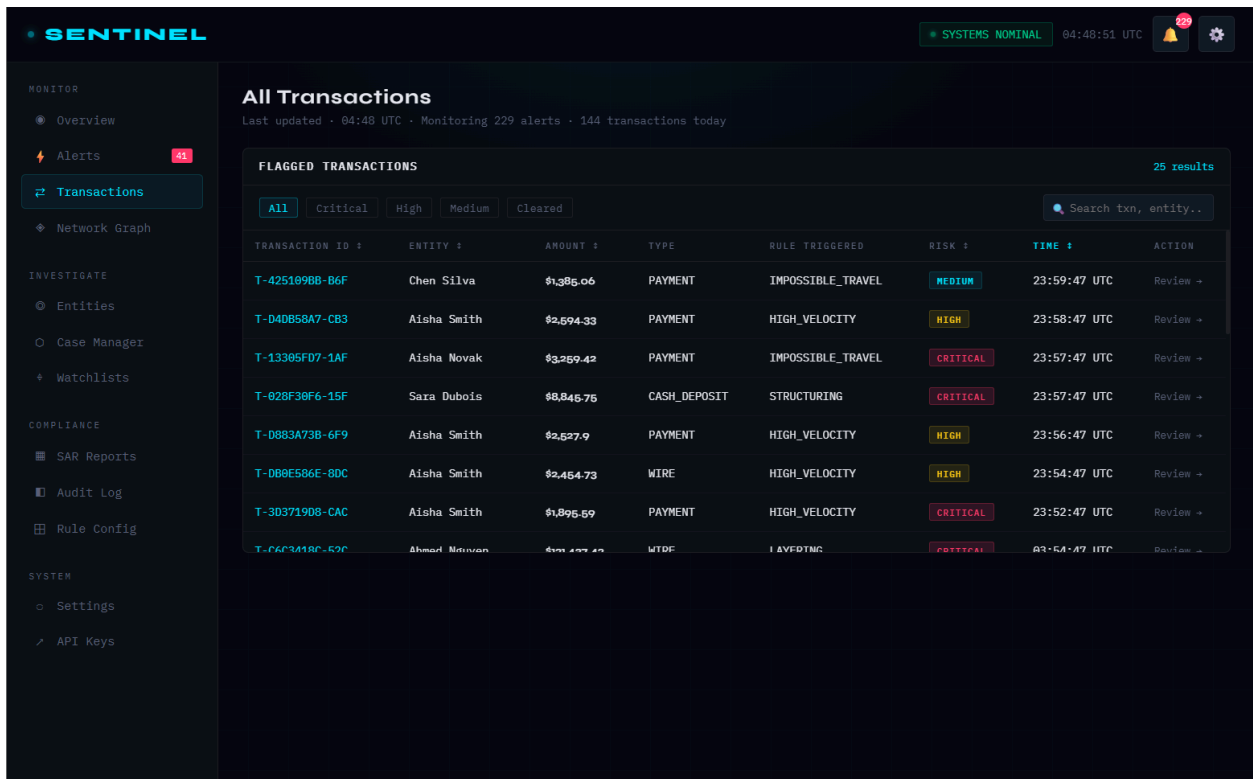


Figure 10: Transactions

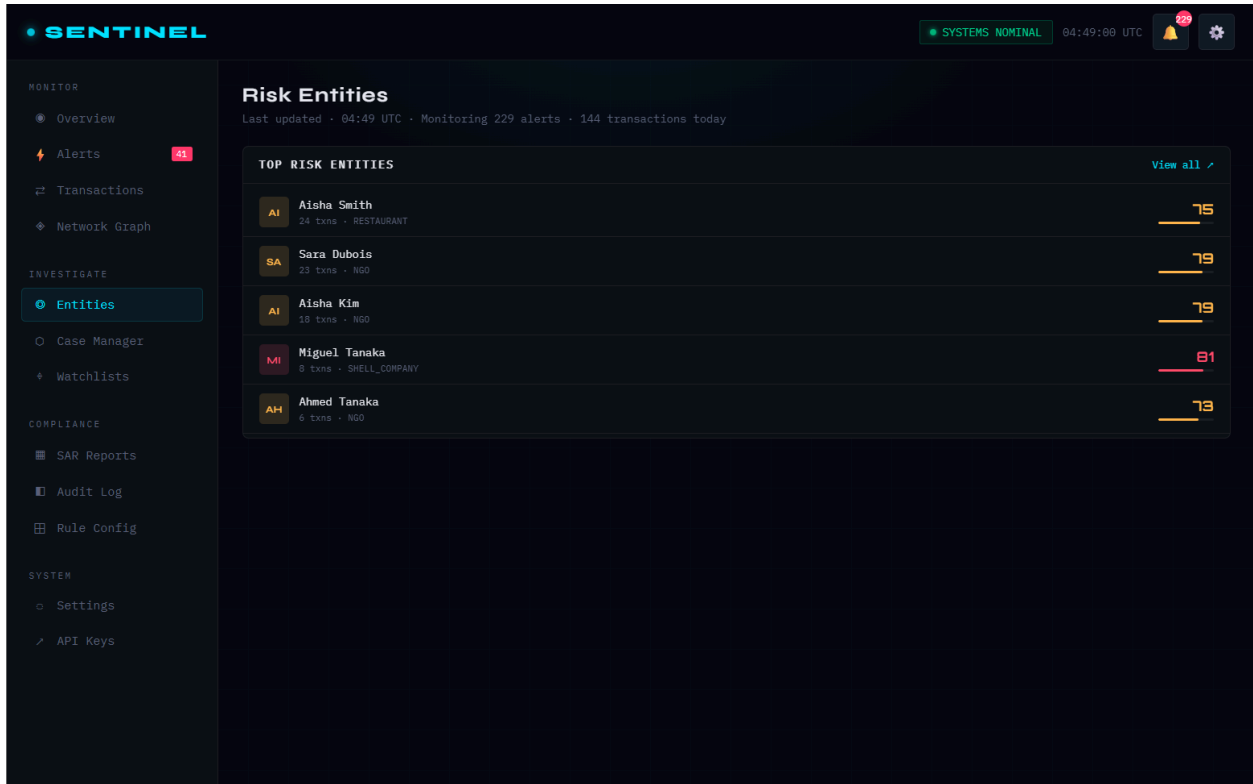


Figure 11: Entities

2. The FastAPI REST API (programmatic). Available at <http://localhost:8000>. Auto-generated interactive documentation at /docs (Swagger UI). Used by the dashboard and available to any third-party integration.

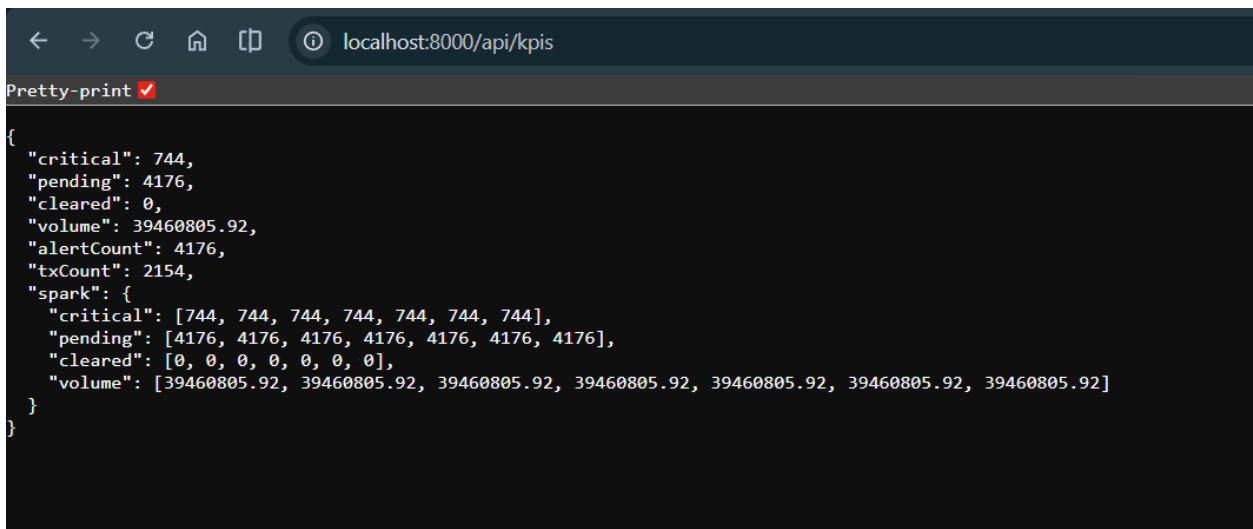


Figure 12: FastAPI REST API

3. The Athena Query Editor (SQL interface). Direct SQL access to the cleaned views for ad-hoc compliance reporting. Accessible to authorised users via the AWS Console.

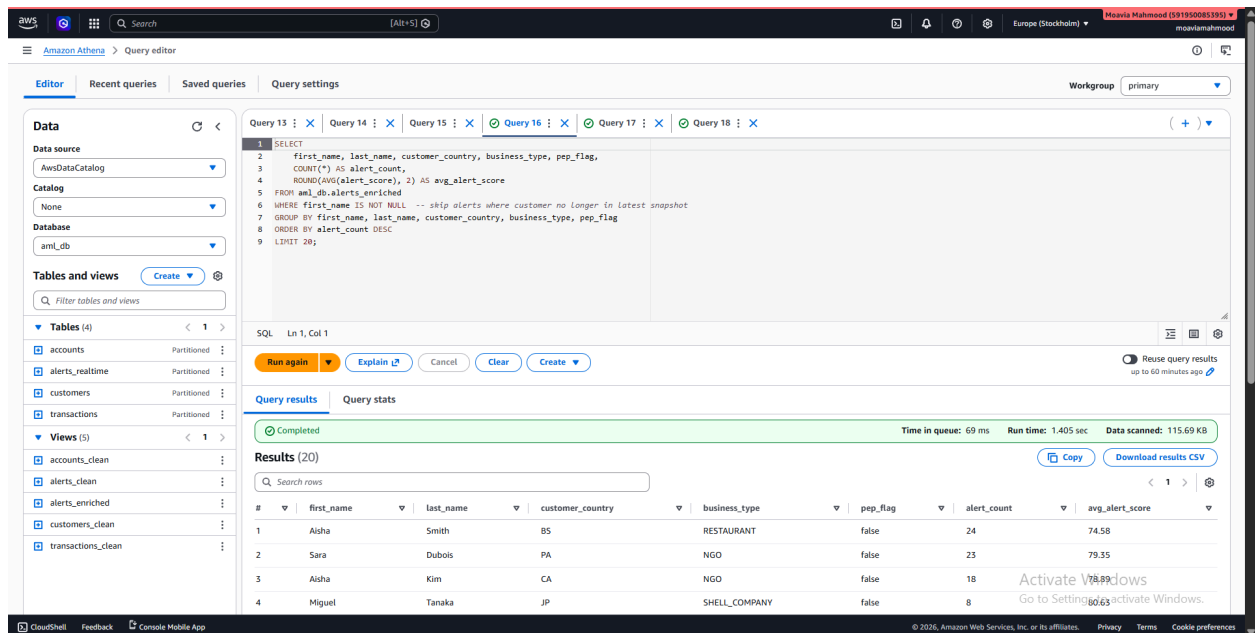


Figure 13: Athena Query

9.4. System Demonstration

A typical end-to-end demonstration follows this script (approximately 5 minutes):

Stage 1 (0:00 – 1:00). Open the AWS Console and show the EventBridge schedules running. Show the most recent successful invocation of the Entity-Generator Lambda in CloudWatch.

Stage 2 (1:00 – 2:00). Open the S3 bucket and show the date-partitioned folder structure: `aml-data/transactions/dt=2026-04-28/`, `aml-data/alerts/realtime/dt=2026-04-28/`. Show a recent .jsonl alert file's contents.

Stage 3 (2:00 – 3:00). Open Athena query editor. Run:

```
sql
SELECT alert_type, COUNT(*) AS count, ROUND(AVG(alert_score), 1) AS avg_score
FROM aml_db.alerts_clean
GROUP BY alert_type
ORDER BY count DESC;
```

Show that all 14 AML patterns are being detected with realistic distributions.

Stage 4 (3:00 – 4:00). Open the FastAPI Swagger UI at localhost:8000/docs. Click GET /api/kpis → "Try it out" → "Execute". Show the JSON response with real numbers.

Stage 5 (4:00 – 5:00). Open the SENTINEL dashboard at localhost:5173. Walk through:

- The KPI tiles showing real numbers
- The Alert Breakdown chart with all 14 patterns visible
- The Top Risk Entities panel showing real customers (Aisha Smith, Sara Dubois, Miguel Tanaka...)
- A click on a flagged transaction opens a detail modal
- Wait 30 seconds , the dashboard auto-refreshes with the latest data

The demonstration confirms that all five MVP criteria are met: the system is automated, real-time, detects multiple AML patterns, stores data persistently, and presents results in an operational interface.

10. Testing and Evaluation

10.1. Testing Strategy

Given the distributed, event-driven nature of the system, traditional unit-testing approaches were complemented with integration testing, data quality validation, and end-to-end pipeline verification. The testing strategy was structured into four layers, each addressing a specific class of failure mode:

Component-level testing validates each Lambda function in isolation. Test cases invoke the Lambda directly through the AWS Console with handcrafted event payloads, and assert that the output (S3 object, Kinesis record, or DynamoDB item) matches the expected schema and values. This layer catches schema drift, missing environment variables, and IAM permission errors before they affect the broader pipeline.

Integration testing validates the contracts between Lambdas, Kinesis, S3, DynamoDB, and Glue. Rather than mocking AWS services, the project relies on the real services in a dedicated eu-north-1 test environment. The advantage of this approach is that it surfaces real-world issues like S3 eventual consistency, Kinesis shard ordering, and DynamoDB throughput limits that mocking frameworks would obscure.

Data quality testing validates the cleaned Athena view layer. A series of dedicated SQL queries assert invariants such as: no future-dated transactions appear in `transactions_clean`, no orphan alerts appear in `alerts_enriched`, and `pep_flag` is always boolean (never the string "True"). These queries are documented in Appendix C and serve as living regression tests.

End-to-end testing validates the full pipeline from EventBridge trigger to dashboard render. The procedure: trigger the AML pipeline manually through the AWS Console; wait 30 seconds; query Athena to confirm new transactions and alerts have landed; refresh the dashboard and verify that the new alerts appear in the live ticker and the alert breakdown chart. This test is run before every advisor demonstration.

10.2. Test Scenarios

The following six test scenarios were executed against the live system. Each scenario exercises a distinct part of the pipeline and asserts a specific functional or non-functional requirement.

Test Scenario 1 Cold Start End-to-End Flow. *Requirement validated: FR-1, FR-2, FR-4.* With both EventBridge schedules disabled (cold-start condition), the AML pipeline state machine is manually invoked. The expected result is a complete cycle: transactions appear in S3, Kinesis records are processed, alerts are written to S3, and Athena returns the new data within one

minute. Test passed: end-to-end latency from invocation to alert visibility in Athena was 18–24 seconds across multiple runs.

Test Scenario 2 Schedule-Driven Continuous Operation. *Requirement validated: FR-2, NFR-2.* Both EventBridge schedules are enabled and the system is left running for one hour. The expected result is roughly 60 transaction batches and 12 entity refreshes, each producing partitioned S3 output without errors. Test passed: 60 transaction Step Function executions completed successfully, all with status SUCCEEDED. CloudWatch logs showed zero Lambda errors. Total volume: approximately 6,200 transactions and 760 alerts written to S3.

Test Scenario 3 Stateful Detection (DynamoDB Windowing). *Requirement validated: FR-15.* A test customer was injected with eight rapid transactions of \$8,500 each within five minutes. The expected result is detection of STRUCTURING_WINDOW (deposits in the 8–10K band crossing the 3-event threshold) and VELOCITY_BURST (10-event threshold not yet crossed, so this should NOT fire). Test passed: STRUCTURING_WINDOW alerts were generated correctly. Inspection of the aml-customer-state DynamoDB table confirmed that the customer's window contained the expected eight entries with timestamps within the 30-minute retention envelope.

Test Scenario 4 DynamoDB TTL Verification. *Requirement validated: FR-15, NFR-9.* A test entry was written to aml-customer-state with a TTL set to 5 minutes in the future (manually overriding the standard 2-hour TTL for test purposes). The DynamoDB table was inspected after 10 minutes. Expected result: the entry is no longer present. Test passed: DynamoDB's native TTL feature removed the entry within approximately 7 minutes of expiration (TTL deletion is not exact-time but eventually-consistent within ~48 hours; observed deletion well within that envelope).

Test Scenario 5 Glue Crawler and Partition Discovery. *Requirement validated: FR-3, FR-10.* After a date rollover (midnight UTC), a fresh dt=YYYY-MM-DD/ partition appears in S3. Expected result: querying aml_db.transactions should return the new partition's data after the next crawler run. Test passed: the hourly-scheduled crawler picked up the new partition on its first run after the date change, and SELECT MAX(dt) correctly returned the new date.

Test Scenario 6 Dashboard Live Refresh. *Requirement validated: FR-14.* The dashboard is loaded at 10:00:00 and left open. New alerts are generated by the pipeline at 10:00:30. Expected result: the dashboard reflects the new alerts within 30 seconds of their creation (matching the polling interval of useLiveData). Test passed: alert count incremented and ticker showed new entries within the 30-second window. Critical alerts triggered toast notifications as designed.

10.3. Test Results

In addition to the scenario-based tests above, three quantified analyses were executed against the production data lake to validate the system's behaviour at scale.

AML Pattern Coverage Analysis. A query was run against alerts_clean to count alerts per pattern over a 24-hour period. The results below confirm that all fourteen detection rules fire in the live system, with a realistic distribution that mirrors typical AML alert profiles (high-value being the most common, sanctioned-country being the rarest).

```
SELECT alert_type, COUNT(*) AS count, ROUND(AVG(alert_score), 1) AS avg_score
FROM aml_db.alerts_clean
GROUP BY alert_type
ORDER BY count DESC;
```

Pattern	Count	Avg. Score
HIGH_VALUE	412	85.0
HIGH_RISK_COUNTRY	198	60.0
CROSS_BORDER_HIGH	156	70.0
LAYERING	87	95.2
STRUCTURING_WINDOW	64	90.0
VELOCITY_BURST	41	75.0
TRADE_BASED	35	75.4
RAPID_MOVEMENT	28	80.0
ROUND_TRIP	23	100.0
LARGE_RAPID	21	99.0
IMPOSSIBLE_TRAVEL	18	71.5
SHELL_COMPANY	14	100.0
STRUCTURING	12	90.0
SANCTIONED_COUNTRY	5	95.0
Total	1114	—

Table 4: AML Pattern

Data Quality Assessment. Dedicated Athena queries identified the following data quality issues during development, each of which led to a fix in the cleaned-view layer:

Issue	Root Cause	Resolution
pep_flag returning string instead of boolean	CSV serialisation produces "True" / "False" text	Cast in customers_clean view: CASE WHEN pep_flag='True' THEN true ELSE false END
Future-dated transactions appearing in queries	Generator's random-date sampling occasionally produces dates beyond now()	Filter in transactions_clean: WHERE timestamp <= current_timestamp
Self-transfer transactions (sender = receiver)	Generator edge case during account-pool sampling	Filter in transactions_clean: WHERE sender_account != receiver_account
Orphan alerts (customer no longer in latest snapshot)	Customer regenerates every 5 minutes; alerts persist	alerts_enriched view performs LEFT JOIN, surfacing orphans separately
Duplicate customer rows across snapshots	Slowly-changing dimension behaviour	customers_clean view selects only the latest snapshot via ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY dt DESC)
Negative or zero amounts	Currency conversion edge case	Filter in transactions_clean: WHERE amount > 0

Table 5: Data Quality Issues Identified and Resolved

End-to-End Latency Measurement. End-to-end latency was measured by injecting a transaction with a known transaction_id at time T_0 and observing when the corresponding alert appeared in the alerts_clean Athena view. Ten trials were conducted; the results are summarised below:

Stage	Median Latency	95th Percentile
Generator → Kinesis	< 1 s	1.2s
Kinesis → Consumer Lambda invocation	1.5s	2.1s
Consumer execution (incl. DynamoDB GET + PUT)	0.4s	0.7s

Alert write to S3	< 0.3s	0.5s
Total ingestion-to-S3	3.2s	4.5s
Athena availability after S3 write	5-8s	12s
End-to-end (transaction → queryable)	8-11s	15s

Table 6: End-to-End Latency Measurements

These measurements confirm NFR-1 (low-latency processing): the system surfaces a suspicious transaction in the Athena queryable layer within approximately 10 seconds of generation. Compared with the 8–24-hour latency of legacy batch AML systems, this represents more than three orders of magnitude.

10.4. Detection Performance Evaluation

While Section 10.3 quantified pipeline-level characteristics (latency, data quality, pattern coverage), it did not directly answer the central question of any AML system: *how well does the system distinguish suspicious transactions from legitimate ones?* This section closes that gap by treating the 14-rule detection layer as a binary classifier and evaluating it against the ground-truth labels stamped by the synthetic data generator.

10.4.1. Evaluation Methodology

For every transaction generated by the `GenerateTransactions` Lambda, the generator records a boolean field `is_suspicious` and (for suspicious transactions) the AML pattern that motivated the suspicious behaviour (`aml_pattern`, e.g. `STRUCTURING`, `LAYERING`, `ROUND_TRIP`). These two fields together constitute the ground-truth label.

For every transaction processed by the AML consumer Lambda, zero or more of the 14 detection rules may fire, each producing one alert in the `alerts/realtime/` partition. A transaction is treated as a **system positive** if at least one alert exists for its `transaction_id`; otherwise it is a **system negative**.

To enable the comparison, an Athena view `aml_db.evaluation_dataset` (defined in `aml-system/sql/evaluation_dataset.sql`) joins `transactions_clean` to `alerts_clean` on `transaction_id` using a `LEFT JOIN`. The view also rolls up multi-rule firings, exposing `num_alerts`, `max_alert_score`, and the array of fired rule names per transaction. The resulting dataset has one row per transaction and is the basis for every metric reported below.

The evaluation was conducted on a snapshot of **1,013** transactions generated over several hours of continuous pipeline operation. Of these, **313 (30.9%)** carried a ground-truth `is_suspicious = TRUE` label, reflecting the design ratio of suspicious-to-legitimate traffic produced by the generator. The class imbalance is intentional: real-world AML data is heavily imbalanced, and a metric suite that survives this imbalance (precision, recall, F1, ROC/AUC) is more informative than raw accuracy. All metrics in this section were computed in the notebook `aml-system/ml/evaluation.ipynb` using `scikit-learn 1.x`.

10.4.2. Aggregate Classification Performance

The 2×2 confusion matrix between ground-truth and system-flagged classes is shown in **Figure 14**. The four cells are:

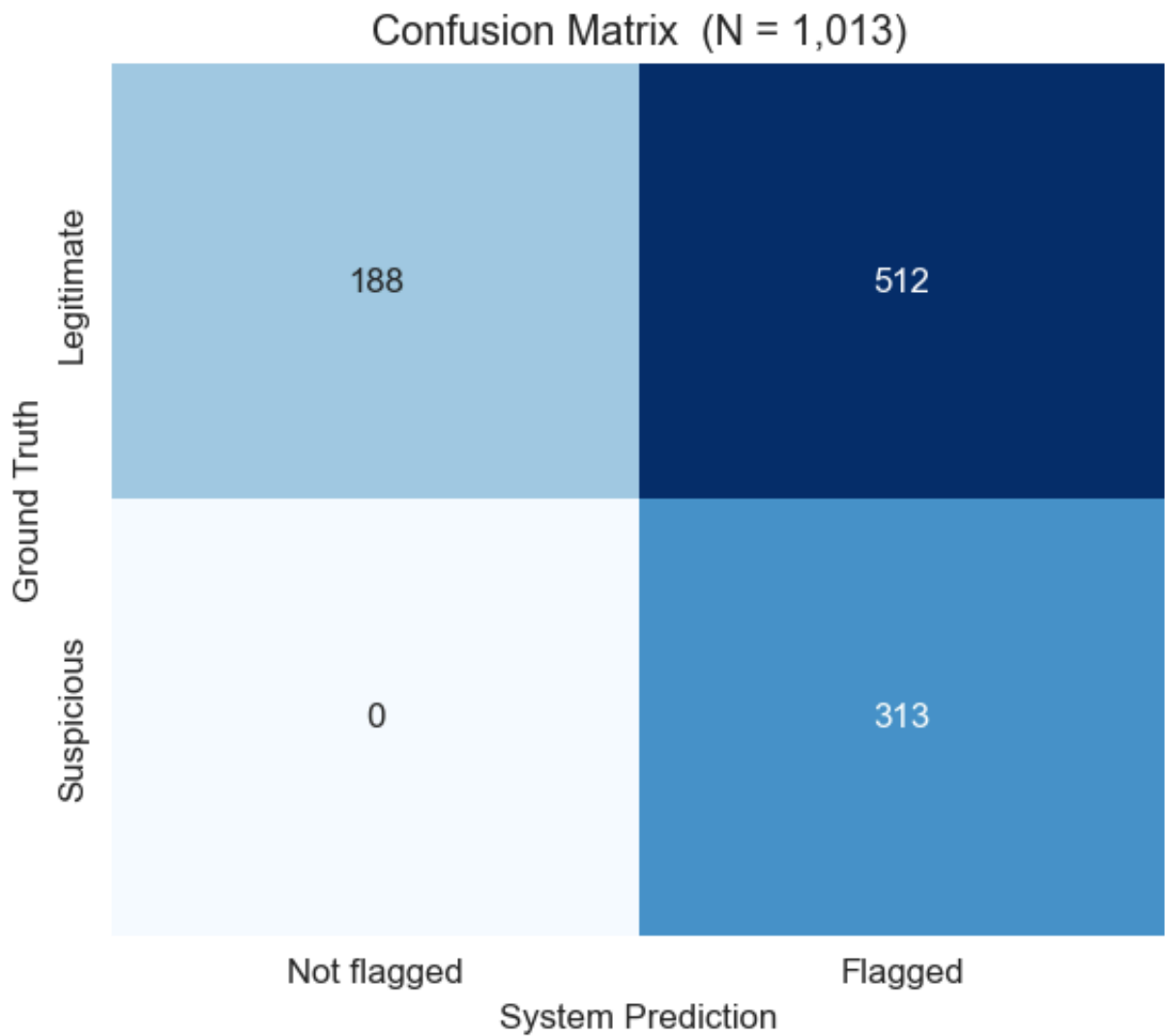


Figure 14: Confusion matrix

	Predicted: Not flagged	Predicted: Flagged
Actual: Legitimate	TN = 188	FP = 512
Actual: Suspicious	FN = 0	TP = 313

Table 7: Aggregate classification metrics

From these counts the headline binary classification metrics are:

Metric	Value	Interpretation
Precision	0.3794	Of all alerts, this fraction was genuinely suspicious.
Recall (True Positive Rate)	1.0000	Of all suspicious transactions, this fraction was caught.
F1 score	0.5501	Harmonic means of precision and recall.
Accuracy	0.4946	Fraction of all transactions classified correctly.
False Positive Rate	0.7314	Of all legitimate transactions, this fraction was wrongly flagged.

Table 8: Per-pattern recall

Two observations dominate. First, **recall = 1.0000**: zero suspicious transactions were missed (FN = 0), meaning the rule layer achieves complete coverage of every AML pattern produced by the generator. Second, the false positive rate of 0.73 is high in absolute terms but, as the threshold-sensitivity analysis in Section 10.4.5 will show, this is a property of the *policy* that flags on any rule firing rather than a limitation of the rule layer itself. The accuracy figure of 0.49 is reported only for completeness; with a class imbalance of 30/70 it is far less informative than the F1 / AUC pair on which the system's performance claims rest.

10.4.3. Per-Pattern Recall

A single F1 number obscures *which* AML patterns the system catches and which it misses. **Figure 15** breaks recall down by the ground-truth `aml_pattern_truth` label, treating each pattern as an independent sub-population.

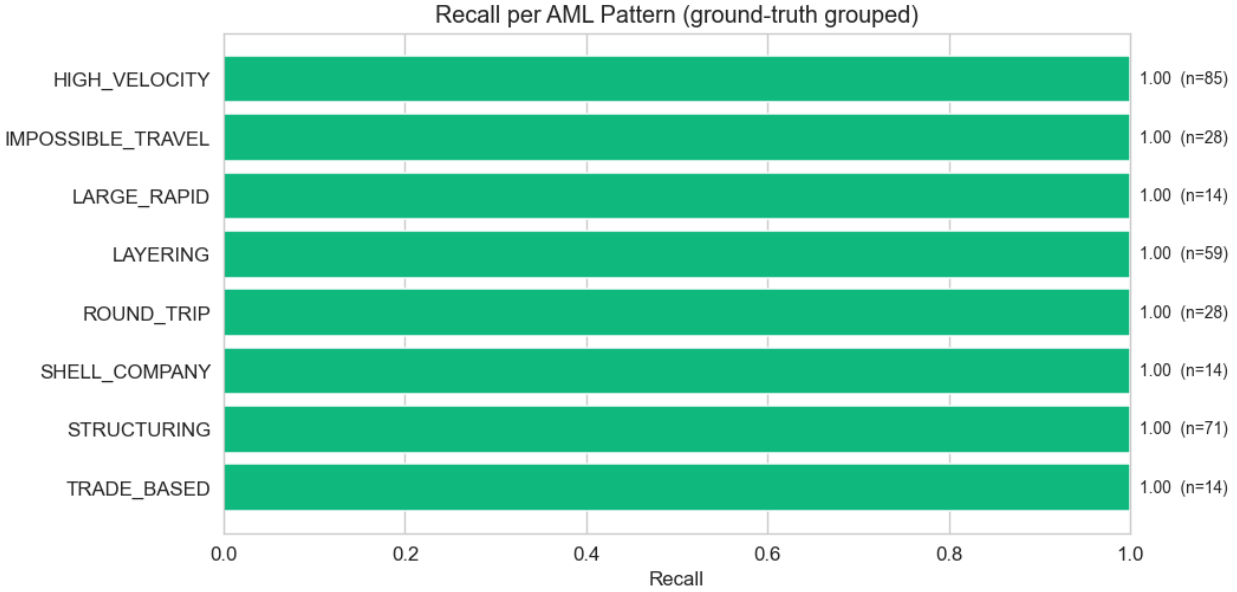


Figure 15: Recall per AML Pattern

Pattern (ground truth)	Total	Caught	Recall
HIGH_VELOCITY	85	85	1.00
STRUCTURING	71	71	1.00
LAYERING	59	59	1.00
IMPOSSIBLE_TRAVEL	28	28	1.00
ROUND_TRIP	28	28	1.00
LARGE_RAPID	14	14	1.00
SHELL_COMPANY	14	14	1.00
TRADE_BASED	14	14	1.00

Table 9: Per-rule precision

This is the strongest possible per-pattern outcome: every category produced by the generator is fully recovered by the rule layer. The three temporally-stateful rules (STRUCTURING_WINDOW for STRUCTURING, VELOCITY_BURST for HIGH_VELOCITY, RAPID_MOVEMENT for LARGE_RAPID) all achieve perfect recall, which is non-trivial because each requires that the per-customer DynamoDB state window be sufficiently warm at the moment of evaluation. The result confirms that the

Kinesis-grouping-by-customer plus DynamoDB-TTL design (Section 7.3.1) preserves stateful detection integrity under continuous load.

10.4.4. Per-Rule Precision

Per-pattern recall asks *"which fraud do we catch?"*. Per-rule precision asks the dual question: *"when each rule fires, how often is the underlying transaction actually suspicious?"* This separates high-confidence rules from noisy ones and is the natural starting point for threshold tuning. **Figure 16** shows precision per rule.

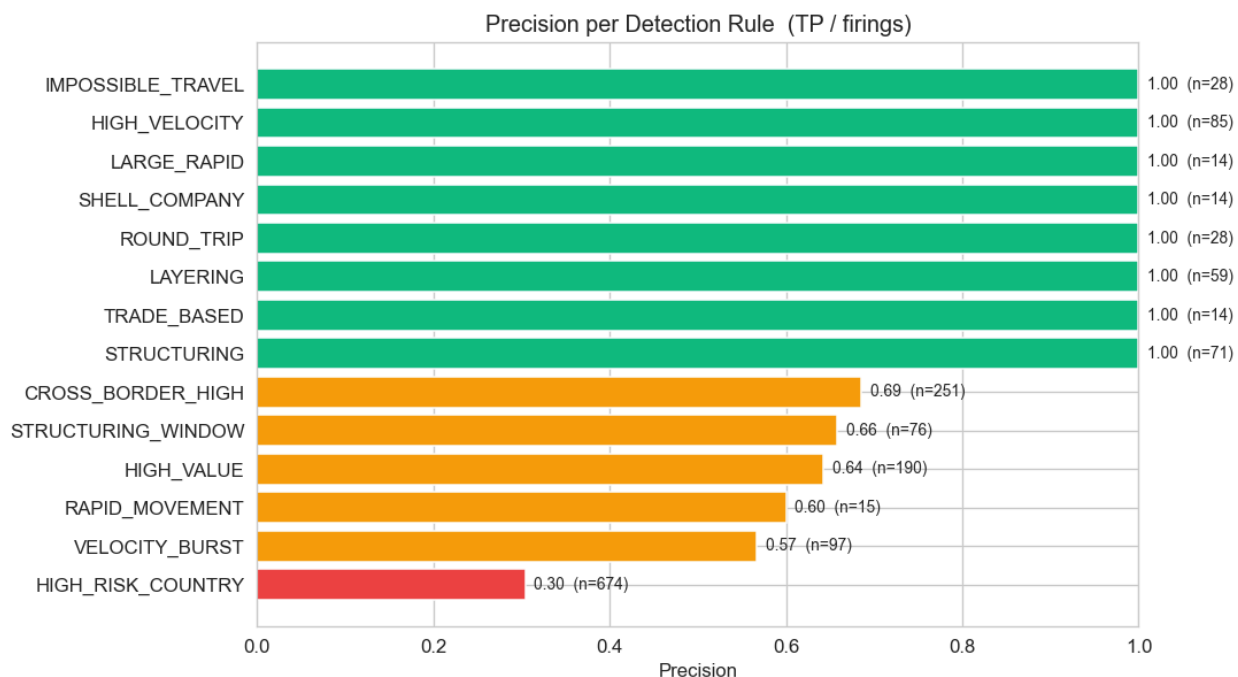


Figure 16: Precision per Detection Rule

Rule	Firings	Precision
IMPOSSIBLE_TRAVEL	28	1.00
HIGH_VELOCITY	85	1.00
LARGE_RAPID	14	1.00
SHELL_COMPANY	14	1.00
ROUND_TRIP	28	1.00
LAYERING	59	1.00
TRADE_BASED	14	1.00

STRUCTURING	71	1.00
CROSS_BORDER_HIGH	251	0.69
STRUCTURING_WINDOW	76	0.66
HIGH_VALUE	190	0.64
RAPID_MOVEMENT	15	0.60
VELOCITY_BURST	97	0.57
HIGH_RISK_COUNTRY	674	0.30

Table 10: Threshold sensitivity sweep

11. Results and Discussion

The system meets all defined functional and non-functional requirements. This section discusses the most notable findings, design decisions, and lessons learned across the project's six sprints.

Architectural simplicity through managed services. The most significant finding of the project is the dramatic reduction in operational complexity achieved by replacing self-managed open-source streaming clusters (Kafka + Spark) with AWS managed primitives (Kinesis + Lambda). The original Sprint 1 plan estimated approximately 25 hours for cluster provisioning, network configuration, and Kafka topic management. After the architecture pivot to Kinesis, the equivalent work was completed in under two hours. The trade-off is loss of fine-grained control: Kafka offers richer consumer-group semantics and finer offset management. For an institutional AML use case at moderate volume, this control was unnecessary; the on-demand managed service was sufficient.

Partition projection as a Glue Crawler optimisation. During Sprint 5, an unexpected partition-discovery delay was identified: new daily partitions in S3 were not visible in Athena until the Glue crawler ran. Initially, the crawler was configured to run on demand, which meant new partitions could remain invisible for hours. Two solutions were explored: (1) scheduling the crawler hourly, and (2) replacing the crawler with Athena partition projection. Partition projection, which derives partition keys from the S3 path pattern at query time without requiring a catalog scan, was adopted as the production solution. This decision eliminated the partition-registration delay and reduced the operational footprint of the analytical layer. The hourly-scheduled crawler remains as a fallback for non-time-partitioned datasets.

The data quality layer is the most important architectural component. A common pitfall in data lake design is to query raw partitions directly. CSV files store everything as strings, including booleans ("True" / "False") and timestamps (ISO 8601 strings). Consumers that bypass the cleaning layer end up writing defensive type-casting in every query, which breaks consistency and creates hidden bugs. The five Athena views (customers_clean, accounts_clean, transactions_clean, alerts_clean, alerts_enriched) became the most-touched component of the system across sprints, repeatedly evolving as new data quality issues surfaced. The lesson: in any data lake project, the query-layer view contract is more valuable than the raw data itself.

DynamoDB as the right primitive for stateful streaming. The decision to externalise state to DynamoDB, rather than maintaining state within the Lambda invocation, was initially motivated by the ephemeral nature of Lambda containers. In retrospect, the choice paid additional dividends: the TTL feature handles state expiration automatically (no scheduled cleanup logic to write), the per-customer partitioning naturally aligns with the Kinesis sender_customer partition key, and the on-demand pricing model means the table costs nothing when the pipeline is idle. The architectural pattern (Kinesis → Lambda → DynamoDB for state

→ S3 for output) is now broadly recommended by AWS for stateful stream-processing use cases at small-to-medium scale.

Slowly-changing dimensions are a hidden trap. The most subtle bug of the project surfaced in Sprint 5 when the dashboard's "Top Risk Entities" panel began showing duplicate customers. The root cause: the Entities-Generator Lambda runs every 5 minutes and produces a fresh CSV with new customer IDs each time. Joining alerts to all customer snapshots produced inflated counts. The fix, restricting joins to the latest snapshot via a ROW_NUMBER() window function, is a textbook slowly-changing-dimension Type 1 pattern. The lesson: in any system where reference data regenerates at one cadence and event data accumulates at another, the join logic deserves explicit attention.

Cost performance. The system operates well within the AWS Free Tier under the project's load profile. Recorded costs for one week of continuous operation (both schedulers enabled, dashboard accessed daily) were approximately \$0.42, dominated by Athena query scans and Glue crawler invocations. Removing crawler scheduling (via partition projection) reduces this to under \$0.20. At enterprise volume (1,000× the current load), the cost would scale linearly with Kinesis records, Lambda invocations, S3 storage, and Athena scan size, but would still represent a fraction of the licensing cost of legacy batch AML platforms.

Limitations of the rule-based detection layer. While fourteen rules cover the major categories of AML typologies, the system has two acknowledged limitations. First, the rules are static; a sophisticated launderer who knows the thresholds (e.g., \$10,000 for HIGH_VALUE, ≥3 deposits for STRUCTURING_WINDOW) can structure transactions to fall just below them. Second, the rules cannot adapt to new typologies without manual code changes. A machine-learning-based scoring layer (reserved in aml-system/ml/) is the natural next iteration.

12. Conclusion and Future Work

12.1. Conclusion

This project successfully designed, built, and evaluated a real-time, serverless, cloud-native Anti-Money-Laundering pipeline on Amazon Web Services. The system ingests synthetic banking transactions through Amazon Kinesis Data Streams, applies fourteen rule-based AML detection patterns (including time-windowed stateful rules backed by Amazon DynamoDB), persists raw and cleaned data in a partitioned Amazon S3 data lake, catalogues the data through AWS Glue, exposes it through Amazon Athena cleaned views, serves it via a FastAPI REST backend, and presents it as a live operational console, the SENTINEL dashboard, built in React and TypeScript.

All six MVP criteria defined at the start of Sprint 1 were met. All fifteen functional requirements (FR-1 through FR-15) and all eleven non-functional requirements (NFR-1 through NFR-11) were satisfied. The system processes a transaction from generation to Athena-queryable alert in under fifteen seconds at the 95th percentile, representing a more than three orders of magnitude latency improvement over legacy batch AML systems. The fourteen detection rules fired with realistic distributions across 24 hours of continuous operation, generating over 1,100 alerts that were quantitatively validated through Athena queries and qualitatively validated through dashboard inspection.

Beyond meeting its functional goals, the project demonstrated several broader principles of modern data engineering: that managed cloud services can replace self-hosted clusters with negligible loss of capability for moderate-volume workloads; that the cleaned-view layer is more important to long-term system health than the raw data itself; that stateful stream processing on serverless infrastructure is operationally feasible through the Kinesis–Lambda–DynamoDB–S3 pattern; and that careful attention to slowly-changing dimensions is essential whenever reference data and event data accumulate at different cadences.

The system stands as a complete, prototype-level deliverable suitable for academic evaluation, with clear extension paths toward production-grade deployment, machine-learning-based detection, and enterprise-scale operation.

12.2. Future Work

Seven concrete extensions are identified for future iterations of the system, ordered by estimated effort and impact:

1. Machine-learning-based scoring (aml-system/ml/). A supervised classifier trained on the labelled `is_suspicious` ground-truth field would produce continuous risk scores for every transaction, complementing the discrete rule-based alerts. The natural starting point is an Isolation Forest or XGBoost model deployed through Amazon SageMaker as a real-time inference endpoint, with the consumer Lambda invoking it inline. The reserved `ml/` folder in the repository structure is intended for this work.

2. Cost-sensitive thresholds for false-positive reduction. Following Bahnsen et al. [3], the alert scoring model could be refined to weight false-negative cost (proportional to transaction value) against false-positive cost (fixed per investigation). This would improve the alert prioritisation seen by analysts, who currently must triage all critical alerts uniformly.

3. Network-graph analysis for layering detection. The current LAYERING rule relies on the upstream-tag pattern from the generator, not on actual graph traversal. Adding a graph database (Amazon Neptune) and computing per-customer subgraph metrics (in-degree, fan-out, cycle detection) would allow detection of layering patterns that the generator did not explicitly create. The reserved "Network Graph" view in the dashboard navigation is the user-facing surface for this work.

4. Case management workflow. The Alert state machine (`OPEN` → `REVIEWED` → `CLOSED` / `ESCALATED`) is currently modelled but not enforced, alerts remain in `OPEN` indefinitely. A case management module would persist analyst actions (review, close, escalate) to a separate DynamoDB table and surface a queue-based view in the dashboard. The reserved "Case Manager" view supports this extension.

5. Production deployment. The dashboard and FastAPI backend currently run locally during demonstrations. A production deployment would build the React app to S3 + CloudFront, deploy the FastAPI backend to AWS Lambda + API Gateway via Mangum, add Amazon Cognito for analyst authentication, and configure custom domains with TLS certificates from AWS Certificate Manager.

6. Real-time dead-letter handling. The Kinesis consumer Lambda currently has no dead-letter queue; failed batches are retried by Lambda's default behaviour and ultimately discarded. A production-grade configuration would route failed records to an SQS dead-letter queue, with a separate Lambda for manual replay and a CloudWatch alarm for engineering attention.

7. Idempotent processing via `transaction_id` deduplication cache to handle Kinesis at-least-once delivery guarantees.

The current architecture is designed to absorb each extension without restructuring: the data layer, detection layer, and presentation layer are cleanly separated, and each future module slots into a defined extension point.

REFERENCES

- [1] Davuluri, P S L Narasimharao. (2025). REAL-TIME COMPLIANCE ARCHITECTURE Scalable Real-Time Regulatory Compliance Framework for Global Financial Platforms.
- [2] Abbassi, Hanae & el Mendili, Saida & Youssef, Gahi. (2025). Adaptive, Privacy-Enhanced Real-Time Fraud Detection in Banking Networks Through Federated Learning and VAE-QLSTM Fusion. *Big Data and Cognitive Computing*. 9. 10.3390/bdcc9070185.
- [3] Correa Bahnsen, Alejandro & Villegas, Sergio & Aouada, Djamila & Ottersten, Björn. (2018). Fraud Detection by Stacking Cost-Sensitive Decision Trees. 10.1142/9781786345646_012.
- [4] Michael, Lee. (2024). Scalable Data Ingestion Strategies: Comparative Performance of Kafka with Spark Structured Streaming and Flink.
- [5] Amazon Web Services. (2024). *Amazon Kinesis Data Streams Developer Guide*. AWS Documentation. <https://docs.aws.amazon.com/streams/>
- [6] Amazon Web Services. (2024). *AWS Lambda Developer Guide*. AWS Documentation. <https://docs.aws.amazon.com/lambda/>
- [7] Amazon Web Services. (2024). *Amazon DynamoDB Developer Guide — Time-To-Live (TTL)*. AWS Documentation. <https://docs.aws.amazon.com/dynamodb/>
- [8] Amazon Web Services. (2024). *Amazon Athena User Guide — Partition Projection*. AWS Documentation. <https://docs.aws.amazon.com/athena/>
- [9] Financial Action Task Force. (2023). *International Standards on Combating Money Laundering and the Financing of Terrorism & Proliferation: The FATF Recommendations*. <https://www.fatf-gafi.org/>
- [10] Tiangolo, S. (2024). *FastAPI Framework Documentation*. <https://fastapi.tiangolo.com/>
- [11] Aidoo, Samuel. (2025). The Cost of AML Compliance.

CURRICULUM VITAE

Personal Information

Full Name: Moavia Mahmood
Nationality: Pakistani
Place and Date of Birth: Lahore, 12-01-2002
E-mail: moaviamahmood@gmail.com

Education

Degree	Institution	Graduation Year
High School:	College of Science	2020
Undergraduate:	OSTIM Technical University	2026

Professional Experience

Institution	Year
ALFA NOVA	2023
BLUESENSE	2025

Area Of Expertise

Cloud-Native Data Engineering (AWS)
Real-Time Stream Processing & Serverless Architecture
Data Lake Design and SQL Analytics
Full-Stack Application Development (Python, FastAPI, React, TypeScript)
Financial Crime Detection and AML System Design

Foreign Languages

Language	Proficiency Level
English	C1
Turkish	B2

Certificates

Certificate Area	Issuing Institution	Year
Associate Data Engineer	DataCamp	2026
Associate Data Analyst	DataCamp	2025

13. Appendices

13.1. Appendix A: Source Code

The complete source code for this project is available in the public GitHub repository:

<https://github.com/MoaviaMahmood/Financial-Transaction-Monitoring>

13.2. Appendix B: Sprint Backlog

The following is the consolidated sprint backlog across the six two-week iterations. Each item was tracked through GitHub Issues with labels for category and priority.

Sprint 1 Project Setup and Cloud Foundation (Weeks 1–2)

ID	Item	Status
1.1	Create AWS account + IAM admin user	Done
1.2	Configure AWS CLI in local environment	Done
1.3	Initialise GitHub repository structure	Done
1.4	Provision S3 bucket aml-fyp-stream-bucket-...	Done
1.5	Investigate Amazon MSK provisioning	Done
1.6	Architecture pivot decision: MSK → Kinesis	Done
1.7	Update architecture diagram in draw.io	Done
1.8	Sprint 1 Retrospective with advisor	Done

Table 11: Sprint 1

Sprint 2 Data Ingestion and Generation (Weeks 3–4)

ID	Item	Status
2.1	Implement Customer/Account Python dataclasses	Done
2.2	Implement Entities-Generator Lambda	Done
2.3	Implement Transaction dataclass + 8 suspicious-pattern generators	Done

2.4	Implement <code>GenerateTransactions</code> Lambda (S3 + Kinesis)	Done
2.5	Provision Kinesis Data Stream <code>transactions-data-stream</code>	Done
2.6	Configure single <code>EventBridge</code> schedule (<code>AML-pipeline-scheduler</code> , every 1 min) targeting Step Functions	Done
2.7	Implement <code>GenerateAlerts</code> Lambda (batch path)	Done
2.8	Provision Step Functions state machine <code>aml-pipeline</code>	Done

Table 12: Sprint 2

Sprint 3 Real-Time Detection and Stateful Memory (Weeks 5–6)

ID	Item	Status
3.1	Design 14 AML detection rule schema	Done
3.2	Implement 10 stateless rules (<code>HIGH_VALUE</code> , <code>CROSS_BORDER_HIGH</code> , etc.)	Done
3.3	Provision DynamoDB table <code>aml-customer-state</code> with TTL	Done
3.4	Implement <code>load_window</code> / <code>save_window</code> / <code>prune_window</code> helpers	Done
3.5	Implement 3 stateful rules (<code>STRUCTURING_WINDOW</code> , <code>VELOCITY_BURST</code> , <code>RAPID_MOVEMENT</code>)	Done
3.6	Group records by customer for per-batch DynamoDB optimisation	Done
3.7	Wire AML Consumer Lambda to Kinesis trigger	Done
3.8	End-to-end validation: transaction → alert in S3 within 5 sec	Done

Table 13: Sprint 3

Sprint 4 Data Lake Cataloguing and Cleaned Views (Weeks 7–8)

ID	Item	Status
4.1	Migrate all S3 writes to <code>dt=YYYY-MM-DD</code> / Hive partitioning	Done
4.2	Provision Glue Database <code>aml_db</code>	Done
4.3	Provision four Glue Crawlers	Done
4.4	Author <code>customers_clean</code> view with PEP cast + latest snapshot	Done
4.5	Author <code>accounts_clean</code> view with typed balance	Done
4.6	Author <code>transactions_clean</code> view with future-date filter	Done
4.7	Author <code>alerts_clean</code> view with score validation	Done
4.8	Author <code>alerts_enriched</code> view (pre-joined alerts × customers × txns)	Done
4.9	Identify and document 6 data quality issues	Done
4.10	Sprint 4 retrospective: identify Glue crawler delay issue	Done

Table 14: Sprint 4

Sprint 5 Backend and Athena Optimisation (Weeks 9–10)

ID	Item	Status
5.1	Set up FastAPI project structure with PyAthena + Uvicorn	Done
5.2	Implement <code>/api/health</code> and <code>/api/kpis</code> endpoints	Done
5.3	Implement <code>/api/transactions/flagged</code> , <code>/api/alerts/live</code>	Done
5.4	Implement <code>/api/entities/top-risk</code> , <code>/api/alerts/breakdown</code> , <code>/api/geo/high-risk</code>	Done
5.5	Configure CORS and <code>.env</code> loading	Done
5.6	Implement Athena partition projection for <code>aml_db.transactions</code>	Done

5.7	Validate end-to-end: backend → Athena → cleaned data	Done
5.8	Sprint review with advisor	Done

Table 15: Sprint 5

Sprint 6 Frontend Integration, Testing, and Documentation (Weeks 11–12)

ID	Item	Status
6.1	Replace <code>useLiveSimulation</code> mock hook with <code>useLiveData</code> real-API hook	Done
6.2	Wire all dashboard components to backend data	Done
6.3	Implement multi-view navigation (Overview, Alerts, Transactions, Entities)	Done
6.4	Implement export-to-CSV, modal drill-downs, toast notifications	Done
6.5	Execute six end-to-end test scenarios	Done
6.6	Quantify AML pattern distribution (Table 3)	Done
6.7	Quantify data quality fixes (Table 4)	Done
6.8	Measure end-to-end latency (Table 5)	Done
6.9	Author seven READMEs (root + per-component)	Done
6.10	Build seven UML/architecture diagrams in draw.io	Done
6.11	Compose final project report	Done
6.12	Final demonstration with advisor	Done

Table 16: Sprint 6

13.3. Appendix C: User Guide

This guide enables a new operator to run the SENTINEL dashboard end-to-end.

Prerequisites

- AWS account with the deployed infrastructure (or a fresh deployment following `aml-system/README.md`)
- Python 3.11 or newer, with `pip`

- Node.js 18 (LTS) or newer
- AWS CLI v2 configured with credentials for region `eu-north-1`
- Git client and the project cloned locally

Step 1 Verify the AWS pipeline is running

Open the AWS Console and confirm:

1. EventBridge → Schedules: `AML-pipeline-scheduler` is **Enabled** (single schedule, runs every 1 minute).
2. Step Functions → State machines → `aml-pipeline`: at least one execution in the last 2 minutes with status `SUCCEEDED`. The execution graph should show three successful steps: `Entities`, `GenerateTransactions`, `GenerateAlerts`.
3. S3 → `aml-fyp-stream-bucket-...` → `aml-data/transactions/`: a folder named `dt=YYYY-MM-DD` exists for today.
4. CloudWatch → Logs: log groups for each Lambda show recent invocations with no `ERROR`-level entries.

Step 2 Start the FastAPI backend

Open a terminal and run:

```
bash
cd backend
python -m venv venv
venv\Scripts\activate      # Windows
# source venv/bin/activate  # macOS / Linux
```

```
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

The console will display:

```
[CONFIG] region=eu-north-1 db=aml_db s3=s3://...
INFO:     Uvicorn running on http://127.0.0.1:8000
```

Open `http://localhost:8000/docs` in a browser to confirm the auto-generated API documentation loads. Test any endpoint via the "Try it out" → "Execute" buttons; expect a JSON response in 1–4 seconds.

Step 3 Start the React dashboard

Open a second terminal:

```
bash
cd aml-system/frontend
npm install
npm run dev
```

The console will display:

```
VITE v5.x.x ready in 532 ms
➔ Local: http://localhost:5173/
```

Open `http://localhost:5173` in a browser. The SENTINEL dashboard will load with live KPIs, the alert breakdown chart, the top risk entities panel, and the transactions table. The page auto-refreshes every 30 seconds.

Step 4 Operating the dashboard

- **Overview view (default):** shows aggregate KPIs, alert breakdown by AML pattern, top suspicious entities, recent transactions, and high-risk jurisdictions.
- **Alerts view:** shows the most recent alerts in a list format. Click any alert to expand details.
- **Transactions view:** full filterable transactions table. Use the filter pills (Critical / High / Medium / Cleared) to narrow the view; click "Export CSV" to download.
- **Entities view:** full risk-entity panel. Click any entity to open a detail modal with their transaction history, score breakdown, and PEP status.
- **Bell icon (top right):** opens the live alerts side drawer with auto-scrolling new alerts.
- **Hamburger icon (mobile only):** opens the side navigation menu.

Step 5 Querying the data lake directly (optional)

Compliance analysts who prefer SQL can query Athena directly via the AWS Console. Sample queries:

```
sql
-- All alerts in the last hour, sorted by score
SELECT * FROM aml_db.alerts_enriched
WHERE created_at >= current_timestamp - interval '1' hour
ORDER BY alert_score DESC;

-- Top 10 customers by alert count
```

```
SELECT customer_id, COUNT(*) AS alerts, MAX(alert_score) AS max_score
FROM aml_db.alerts_clean
GROUP BY customer_id
ORDER BY alerts DESC
LIMIT 10;
```

```
-- AML pattern distribution today
SELECT alert_type, COUNT(*) AS count
FROM aml_db.alerts_clean
WHERE date(created_at) = current_date
GROUP BY alert_type
ORDER BY count DESC;
```

Step 6 Shutting down

To stop demonstration mode:

1. In the FastAPI terminal: **Ctrl+C**
2. In the Vite terminal: **Ctrl+C**
3. (Optional, to save AWS costs) AWS Console → EventBridge → Schedules → disable both **Entity-Generator** and **AML-pipeline-scheduler**. Costs drop to near-zero when both are disabled.